西南交通大學
Southwest Jiaotong University

《软件系统综合课程设计》测试报告

设计题目： 西南交通大学校园论坛系统综合测试

设计人员： 2023112621—程泽凯

2023112592—赵祺

2023114956—金井直

2023112548—蒋焰

班　　级： 计算机 2023 级

指导教师： 李威、殷成风

2026 年 7 月

摘要

本报告以“西南交通大学校园论坛系统”为统一被测对象，整合软件系统综合课程设计练习 1、练习 2 和练习 3 的测试工作。测试活动依次覆盖系统外部功能、后端代码逻辑和运行时性能三个层次。练习 1 采用等价类划分、边界值分析、判定表和场景法，对注册、登录、发帖、评论、互动、成长系统、积分商城及通知中心等 15 个功能点实施黑盒与功能测试；练习 2 采用基本路径法、条件–判定覆盖和条件组合覆盖，对服务层、控制层及工具类中的 12 个代码单元进行 JUnit 5 自动化测试；练习 3 采用负载、压力、容量和稳定性测试，对公共查询接口的吞吐量、响应时间分位数和错误率进行测量。现有执行记录显示，功能测试共执行 75 条用例，其中 70 条通过、5 条失败；后端 Maven 测试共执行 85 项，失败、错误和跳过均为 0；性能测试各场景错误率均为 0。报告同时记录 6 项功能缺陷，其中 3 项已完成修复与回归验证。综合结果表明，系统核心业务链路与后端关键逻辑具备较好的可用性，但仍需持续关注中文编码、业务数据一致性、身份信息覆盖确认以及生产环境下的长期性能表现。

关键词：校园论坛；黑盒测试；白盒测试；JUnit；性能测试；缺陷管理

目录

一、 项目与测试任务概述 1

1.1 项目简介 1

1.2 测试范围与边界 1

1.3 成员分工 1

二、 测试环境与执行规范 2

2.1 软硬件环境 2

2.2 结果判定原则 2

2.3 测试过程与证据管理 2

2.4 需求与测试追踪 2

三、 练习 1：黑盒与功能测试 4

3.1 测试设计方法 4

3.2 等价类与边界值分析 4

3.3 判定表设计 5

3.4 功能点与代表性测试 6

3.5 代表性测试用例分析 7

3.6 场景链路分析 9

3.7 执行结果与缺陷 10

3.8 页面与回归证据 11

四、 练习 2：白盒单元测试 13

4.1 测试对象与方法 13

4.2 各代码单元的测试设计 14

4.2.1 用户、认证与分页单元 15

4.2.2 帖子搜索、发布与评论单元 15

4.2.3 身份认证、签到与兑换单元 15

4.2.4 后台治理、举报与通知单元 16

4.3 控制流与基本路径分析 16

4.4 条件组合分析 17

4.5 自动化执行结果 17
五、 练习 3：性能测试 18
5.1 场景设计与统计口径 18
5.2 性能指标定义 18
5.3 原始结果汇总 19
5.4 负载测试分析 19
5.5 压力测试趋势 20
5.6 容量与稳定性分析 21
5.7 性能结论的适用范围 22
六、 综合质量评价与改进建议 22
6.1 测试结论 22
6.2 缺陷优先级与回归策略 22
6.3 自动化与持续集成建议 23
七、 实验总结 24
A 自动化脚本清单 25
1.1 Selenium 黑盒测试脚本 25
B JUnit 白盒测试脚本 29
2.1 用户服务测试 29
2.2 认证控制器测试 34
2.3 分页工具测试 37
2.4 成长与发现服务测试 37
2.5 帖子服务测试 46
2.6 帖子控制器测试 55
2.7 后台管理服务测试 58
2.8 举报服务测试 61
2.9 通知服务测试 69
C 性能测试脚本 71
D 原始结果文件说明 75

一、 项目与测试任务概述

1.1 项目简介

西南交通大学校园论坛系统面向校内信息交流场景，采用前后端分离架构。后端基于 Spring Boot 实现业务接口，数据访问层使用 Spring Data JPA；前端承担页面渲染、交互与接口调用。系统业务范围包括用户注册与登录、帖子浏览与搜索、发帖与评论、点赞与收藏、个人资料维护、校园身份认证、每日签到、积分兑换、通知查看、用户关注以及后台内容治理等功能。

综合测试的目标不是简单汇总三次练习，而是从用户行为、程序结构和运行质量三个角度形成连续证据。功能测试回答“系统能否按需求完成业务”；白盒测试回答“关键判断与异常分支是否被验证”；性能测试回答“在不同请求规模下，系统是否保持可接受的响应与稳定性”。三类结果共同构成对系统当前版本质量的评价依据。

1.2 测试范围与边界

本报告覆盖论坛 Web 端主要业务流程、后端关键服务方法和无需登录的公共查询接口。功能测试以便携运行包和浏览器页面为对象；单元测试以 `jiaodaluntan/backend` 中的 Java 代码为对象；性能测试以`/api/posts`、`/api/posts/hot`、`/api/posts/search` 和`/api/sections` 等公共接口为对象。报告不将短时本机压测结果等同于生产环境容量上限，也不在未启用 JaCoCo 的情况下推测代码覆盖率百分比。

1.3 成员分工

四名成员在统一项目、统一缺陷口径和统一执行环境下分别承担不同测试重点。分工既覆盖练习 2 的代码单元，也覆盖练习 3 的四类性能场景，具体见表1。

表 **1**　综合测试分工

成员	白盒测试重点	性能测试重点
金井直	用户注册、账号密码登录、分页参数修正	常规混合浏览负载测试
蒋焰	帖子搜索入口、帖子发布、评论发布	并发阶梯压力测试
赵祺	校园身份认证、每日签到、积分兑换	分页容量测试
程泽凯	后台封禁用户、举报创建、通知已读	短时持续稳定性测试

二、 测试环境与执行规范

2.1 软硬件环境

测试主机运行 Windows 11，处理器逻辑核心数为 16，内存约 31.31 GiB。后端使用 Java 17、Spring Boot、Maven、JUnit 5 与 Mockito，前端功能自动化使用 Selenium WebDriver。性能脚本运行于 Node.js 环境，并通过高精度计时接口记录每个请求的完成时间。被测接口基址为 `http://127.0.0.1:18081/api`。相关部署信息还记录了 Ubuntu 22.04、2 核 CPU、2 GiB 内存、40 GiB 磁盘和 200 Mbps 带宽的服务器配置，但本轮原始性能数据来自本机接口地址，因此报告将其解释为本机环境观测值。

2.2 结果判定原则

功能测试以页面状态、接口响应、数据持久化结果和业务提示是否符合需求为判定依据。单元测试以断言结果、异常类型、返回对象和 Repository 交互为判定依据。性能测试以成功率、错误率、吞吐量、平均响应时间以及 P95、P99 分位数为主要指标。所有结果均引用现有执行文件；对未执行的长时间稳定性、真实广域网并发和数据库极限容量不作外推。

2.3 测试过程与证据管理

综合测试按照“需求分析、测试设计、环境准备、测试执行、缺陷记录、修复验证、结果归档”的顺序实施。需求分析阶段首先将系统功能拆分为可独立验证的测试点，再识别每个功能点的输入、前置状态和可观察输出。测试设计阶段根据对象特征选择黑盒、白盒或性能方法，并提前确定通过标准。执行阶段保留用例表、Surefire 报告、性能 JSON 和页面截图，使结论能够回溯到原始证据。

缺陷管理采用编号、模块、严重程度、现象、状态和证据六项基本信息。对于阻断主流程或导致 500 响应的问题，修复后必须重新执行原失败步骤，并补充正常页面或接口结果。对于暂未修复的问题，报告不将其改写为“已通过”，而是在综合结论中保留风险。该处理方式使报告既反映系统当前能力，也明确区分已经验证的事实与后续改进建议。

2.4 需求与测试追踪

表2建立需求、测试层次和证据之间的对应关系。功能需求主要由练习 1 验证，内部业务规则由练习 2 补充，读取性能由练习 3 观察。一个需求可能同时由多类测试覆盖，例如积分兑换既需要黑盒检查用户操作，也需要单元测试验证积分和库存条件，还需要在回归中确认兑换记录接口不再返回 500。

表 **2**　需求–测试追踪矩阵

编号	需求模块	主要质量风险	测试证据	测试层次
R01	注册与登录	重复账号、错误凭据、登录态未建立	注册与登录用例、认证控制器测试	功能/单元
R02	帖子浏览	分页错误、空数据异常、列表不可见	首页列表用例、分页工具测试	功能/单元
R03	版块与搜索	条件未生效、特殊关键字导致异常	筛选与搜索用例、控制器测试	功能/单元
R04	发帖与评论	未登录写入、空内容、关联对象缺失	发帖评论用例、帖子服务测试	功能/单元
R05	点赞与收藏	重复操作、计数不同步、刷新丢失	互动用例及页面截图	功能
R06	个人资料	非法字段写入、保存后状态不一致	资料编辑用例与回归截图	功能
R07	校园认证	格式错误、重复认证、覆盖可信资料	认证用例、用户服务测试	功能/单元
R08	每日签到	重复加分、签到记录与流水不一致	签到用例、成长服务测试	功能/单元
R09	积分兑换	积分或库存不足仍成功、记录加载失败	兑换用例、服务测试、缺陷截图	功能/单元
R10	通知中心	面板存在但不可见、归属校验错误	通知用例、通知服务测试、截图	功能/单元
R11	用户关注	重复关注、关注自己、取消失败	关注与取消关注用例	功能

编号	需求模块	主要质量风险	测试证据	测试层次
R12	后台治理	越权封禁、受保护角色被操作	后台管理服务测试	单元
R13	举报处理	原因非法、重复举报、帖子不存在	举报服务测试	单元
R14	公共查询性能	并发升高导致延迟或错误率上升	压力与稳定性原始 JSON	性能
R15	分页容量	大页请求导致吞吐量下降	容量测试数据与趋势图	性能

三、 练习 1：黑盒与功能测试

3.1 测试设计方法

练习 1 以需求和用户可见行为为输入，不依赖内部实现细节。等价类划分用于区分合法输入、非法输入、未登录状态、权限不足、资源不存在和重复操作；边界值分析用于检查用户名与密码长度、分页参数、积分余额、学号格式和空文本等临界情况；判定表用于表达登录状态、资源状态、积分是否充足及操作是否重复等多条件组合；场景法用于连接注册、登录、发帖、评论、通知和积分等跨模块业务链路。

3.2 等价类与边界值分析

输入域并非只按“正确”和“错误”划分。以注册为例，有效等价类包括未被占用的账号、满足格式约束的密码和完整必填信息；无效等价类包括重复账号、空字段、长度越界和格式非法。对帖子与评论，除了空文本和正常文本，还需要考虑仅包含空白字符、特殊符号及较长内容。对登录后功能，则将“已登录”和“未登录”视为两个不同状态类，因为相同输入在不同会话状态下应得到不同结果。

边界值重点放在分页、文本长度、积分余额和库存数量。分页参数需覆盖负页码、零页码、正常页长和过大页长；积分兑换需覆盖余额小于、等于和大于商品所需积分；库存需覆盖 0 和 1 两个临界值；每日签到需覆盖当天首次操作与当天重复操作。表3汇总主要输入域及预期处理。

表 **3**　主要等价类与边界值

输入或状态	有效类	无效类或边界	预期处理
注册账号	未占用且格式合法	空值、重复值、长度越界	成功创建或明确拒绝
登录凭据	账号存在且密码匹配	账号不存在、密码错误、空输入	不建立登录态
帖子标题与正文	非空且满足长度要求	空白文本、超长文本、未登录	阻止发布并提示原因
搜索关键字	普通文字与可接受符号	空关键字、无匹配、特殊字符	返回列表或空结果，不应 500
分页参数	页码非负、页长在限制内	负页码、过大页长	修正或拒绝非法参数
签到状态	当天尚未签到	当天已经签到	仅首次增加积分
积分余额	余额大于或等于兑换值	差 1 分、余额为 0	成功兑换或保持原余额
商品库存	库存大于 0	库存为 0、商品下架	不生成兑换记录
通知归属	通知属于当前用户	通知不存在或属于他人	更新已读或拒绝访问
关注关系	未关注其他有效用户	关注自己、重复关注、目标不存在	建立一次关系或拒绝

3.3 判定表设计

积分兑换由登录状态、商品状态、库存和积分四项条件共同决定。只有四项均满足时，系统才允许扣减积分、减少库存并生成兑换记录。任何前置条件不满足都应终止后续写入，且失败后用户积分和商品库存不得变化。表4给出精简判定表，其中“–”表示该条件在前序条件失败后不再影响结果。

表 **4**　积分兑换判定表

条件或动作	规则 1	规则 2	规则 3	规则 4	规则 5
用户已登录	否	是	是	是	是
商品可兑换	–	否	是	是	是
库存充足	–	–	否	是	是
积分充足	–	–	–	否	是
返回失败提示	是	是	是	是	否
扣减积分与库存	否	否	否	否	是
生成兑换记录	否	否	否	否	是

3.4 功能点与代表性测试

功能测试覆盖 15 个功能点，每个功能点保留 5 条高价值用例，共 75 条。表5展示各功能点的主要验证目标，完整用例数据保存在原始测试用例文档中。

表 **5**　黑盒功能测试点汇总

序号	功能点	正常流程	重点异常或边界
1	用户注册	合法账号完成注册并进入系统	重复账号、空字段、长度边界与非法格式
2	用户登录	正确账号密码建立登录态	错误密码、未知账号、连续失败与空输入
3	首页帖子列表	正常加载帖子与分页信息	空列表、分页边界、刷新后状态一致性
4	版块筛选	按指定版块返回对应帖子	不存在版块、无帖子版块与筛选切换
5	帖子搜索	关键字返回匹配内容	空关键字、无匹配结果与特殊字符
6	发布帖子	合法标题与正文发布成功	空标题、空正文、未登录与超长文本

序号	功能点	正常流程	重点异常或边界
7	发表评论	登录用户成功发表评论	空评论、未登录、目标帖子不存在
8	点赞帖子	点赞后状态与计数更新	重复点击、取消点赞与刷新持久化
9	收藏帖子	收藏后个人中心可查询	重复收藏、取消收藏与未登录访问
10	编辑资料	修改昵称等资料并保存	空值、长度边界与刷新后一致性
11	校园身份认证	合法身份信息认证成功	学号格式错误、重复认证与覆盖确认
12	每日签到	首次签到增加积分并生成记录	当日重复签到与积分流水一致性
13	积分兑换	积分充足时兑换成功	积分不足、库存不足与重复提交
14	通知中心	打开面板并查看通知内容	空通知、未读状态与面板可见性
15	关注用户	关注和取消关注均正确生效	关注自己、重复关注与目标不存在

3.5 代表性测试用例分析

为避免测试用例只停留在功能名称层面，表6进一步列出每个功能点的一条正常用例和一条高风险异常用例。执行时除观察页面提示外，还需要检查刷新后的状态、列表中的数据以及关联模块是否同步变化。例如点赞成功后既要观察按钮状态，也要确认计数变化；签到成功后既要检查积分总额，也要检查签到记录和积分流水。

表 **6**　代表性功能测试用例

编号	测试点	前置条件与输入	预期结果	类型
F01-1	用户注册	未登录，输入未占用账号及合法资料	注册成功，建立用户数据并进入可登录状态	正常
F01-2	用户注册	输入已经存在的账号	拒绝注册，原账号资料不被覆盖	异常

编号	测试点	前置条件与输入	预期结果	类型
F02-1	用户登录	使用已注册账号和正确密码	登录成功，页面显示当前用户信息	正常
F02-2	用户登录	使用正确账号和错误密码	明确提示失败，不生成有效登录态	异常
F03-1	首页列表	服务正常且存在帖子数据	加载帖子、作者、版块和分页信息	正常
F03-2	首页列表	请求超出有效范围的页码	返回空页或修正参数，不出现 500	边界
F04-1	版块筛选	选择存在且有帖子的版块	仅展示该版块下符合条件的帖子	正常
F04-2	版块筛选	选择当前无帖子的版块	展示空状态，不残留上一版块数据	异常
F05-1	帖子搜索	输入能够命中标题或正文的关键字	返回匹配帖子并保留查询条件	正常
F05-2	帖子搜索	输入无匹配结果的关键字	展示空结果，不将全部帖子误作结果	异常
F06-1	发布帖子	已登录，选择版块并填写合法标题正文	发布成功并进入新帖详情页	正常
F06-2	发布帖子	未登录或标题为空	阻止写入并提示登录或字段要求	异常
F07-1	发表评论	已登录，在有效帖子下输入合法评论	评论写入并出现在评论列表	正常
F07-2	发表评论	输入空白评论或目标帖子不存在	不保存评论并返回明确提示	异常
F08-1	点赞帖子	已登录且尚未点赞目标帖子	点赞状态与计数同步更新	正常
F08-2	点赞帖子	连续执行点赞与取消操作	关系不重复，计数不出现累计偏差	状态
F09-1	收藏帖子	已登录且尚未收藏目标帖子	收藏成功，个人收藏列表可查询	正常
F09-2	收藏帖子	取消收藏后刷新详情与列表	收藏状态保持取消，不出现旧状态	状态
F10-1	编辑资料	已登录，修改允许编辑的个人字段	保存成功，刷新后仍显示新数据	正常

编号	测试点	前置条件与输入	预期结果	类型
F10-2	编辑资料	提交空昵称或不合法长度	拒绝非法值，原有效资料保持不变	边界
F11-1	校园认证	提交匹配的学号和校园邮箱	认证成功并设置可信身份状态	正常
F11-2	校园认证	邮箱与学号不匹配	认证失败，原可信字段不被覆盖	异常
F12-1	每日签到	当天尚未签到的登录用户执行签到	增加一次积分并生成签到记录	正常
F12-2	每日签到	同一用户当天再次签到	不重复增加积分，提示已经签到	状态
F13-1	积分兑换	商品有效、库存充足且积分足够	扣分、减库存并生成兑换记录	正常
F13-2	积分兑换	用户积分比所需积分少 1 分	兑换失败，积分和库存均不变化	边界
F14-1	通知中心	登录用户点击通知入口	面板真实可见并展示通知列表	正常
F14-2	通知中心	通知 DOM 存在但样式仍为隐藏	用例判定失败，不能仅以 DOM 存在通过	异常
F15-1	关注用户	已登录且目标为其他有效用户	建立关注关系，按钮变为取消关注	正常
F15-2	关注用户	尝试关注自己或重复关注	拒绝非法关系，不生成重复记录	异常

3.6 场景链路分析

论坛业务具有明显的状态依赖。新用户只有在注册并登录后才能发帖、评论和互动；发帖或评论可能进一步产生通知；签到获得的积分影响积分商城兑换结果。因此，测试执行不仅检查单个按钮，还通过“注册–登录–发帖–评论–通知”和“登录–签到–积分变化–商品兑换”两条链路验证状态传递。该设计能够发现仅靠孤立接口测试不易暴露的刷新、持久化和前端可见性问题。

第一条链路从新用户注册开始。注册成功后使用相同凭据登录，进入首页确认登录态，再发布一篇测试帖子。随后进入详情页发表评论并执行点赞、收藏操作，最后检查相关用户的通知中心。链路中的每一步都以上一步产生的数据为输入，因此能够同时验证用户标识、帖子标识、评论关系和通知关系是否正确传递。若中途刷新页面后状态丢

失，则说明前端展示或持久化存在缺陷，即使单次按钮操作曾显示成功，也不能判定整个场景通过。

第二条链路围绕成长与积分业务展开。用户进入成长中心执行当天首次签到，记录签到前后的积分值，再进入积分商城选择可兑换商品。兑换前核对商品所需积分与库存，兑换后检查用户积分、商品库存和兑换记录。若重复签到仍加分，或兑换记录刷新导致500，则链路失败。该场景把三个独立页面连接起来，可以验证积分作为跨模块共享数据时的一致性。

表 **7**　跨模块场景检查点

场景	关键步骤	主要检查点	失败判定
内容互动链路	注册、登录、发帖、评论、点赞、收藏、通知	登录态连续，帖子与互动关系持久化，通知面板可见	任一步骤数据丢失、越权成功或通知不可见
成长兑换链路	登录、签到、积分核对、商品兑换、记录刷新	仅首次签到加分，兑换原子更新，记录可重新加载	重复加分、积分库存不同步或记录接口异常

3.7 执行结果与缺陷

现有执行摘要生成于 2026 年 6 月 9 日，共设计并执行 75 条用例，其中 70 条通过、5 条失败，无阻塞和不适用用例。执行期间记录 6 项缺陷，3 项已修复并完成回归，另外 3 项处于待修复或待优化状态。测试失败主要集中在通知面板可见性、积分兑换记录刷新、中文显示与数据一致性等非主流程问题，核心注册、登录、浏览和发帖链路可正常完成。

表 **8**　功能测试执行统计

统计项	执行总数	通过	失败	阻塞
用例数量	75	70	5	0
占比	100%	93.33%	6.67%	0%

表 **9**　功能测试缺陷清单

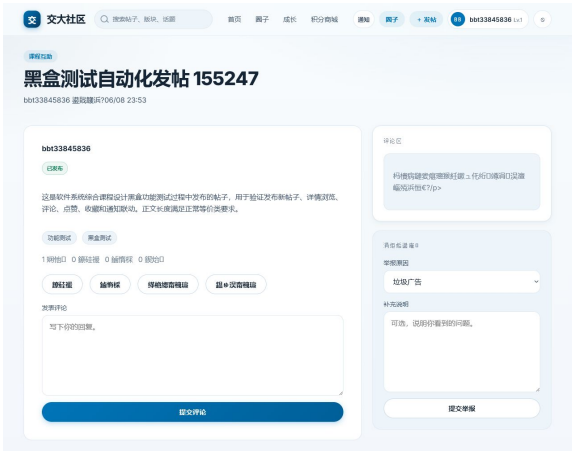
编号	模块	严重程度	缺陷描述	状态
BUG-001	通知中心	高	点击通知后面板 DOM 存在但不可见，用户无法完成查看操作	已修复
BUG-002	积分商城	高	兑换记录接口刷新出现 500，导致页面加载失败	已修复
BUG-003	便携包启动	高	旧 H2 数据库升级时非空字段缺少默认值，导致应用启动失败	已修复
BUG-004	发帖	中	部分详情页按钮或静态文案出现中文乱码	待修复
BUG-005	成长系统	中	签到记录积分展示与积分流水不完全一致	待修复
BUG-006	个人中心	低	校园身份认证覆盖学院与专业信息前缺少二次确认	待优化

3.8 页面与回归证据

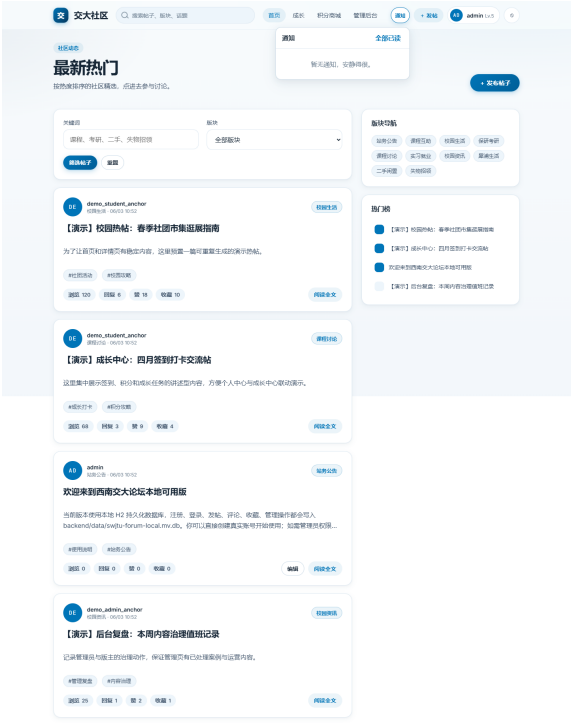
图1展示注册、发帖、通知和积分商城的页面证据。这些截图用于证明测试对象真实运行，并为缺陷复现与回归提供可追溯依据。



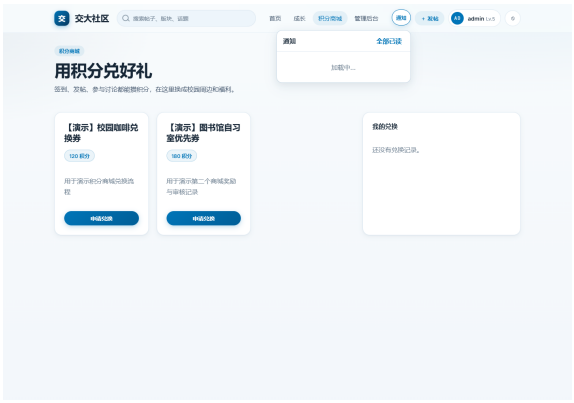
(a) 注册成功后进入首页



(b) 帖子发布成功详情页



(c) 通知面板缺陷回归证据



(d) 兑换记录缺陷回归证据

图 **1**　练习 **1** 功能测试页面证据

四、 练习 2：白盒单元测试

4.1 测试对象与方法

白盒测试从后端源码中选取 12 个具有条件判断、异常分支或提前返回的代码单元。基本路径法用于依据控制流图计算环形复杂度并规划独立路径；条件–判定覆盖用于保证每个判定结果和主要原子条件至少取到一次真值与假值；条件组合覆盖用于验证多个业务条件共同决定结果的场景。JUnit 5 负责测试组织和断言，Mockito 用于隔离 Repository 及外部依赖。

表 **10**　白盒测试单元与覆盖重点

成员	代码单元	主要分支	对应测试脚本
金井直	用户注册逻辑	账号重复、字段合法、保存成功	`UserServiceTest.java`
金井直	账号密码登录	用户存在、密码匹配、认证成功	`AuthController Test.java`
金井直	分页参数修正	负页码、过大页长与正常参数	`PageablesTest.java`
蒋焰	帖子搜索入口	关键字为空、分页修正与正常搜索	`PostController Test.java`
蒋焰	帖子发布逻辑	用户与版块存在、内容合法、保存成功	`PostServiceTest.java`
蒋焰	评论发布逻辑	帖子存在、评论合法与通知触发	`PostServiceTest.java`
赵祺	校园身份认证	信息格式、重复认证与状态更新	`UserServiceTest.java`
赵祺	每日签到逻辑	当日是否签到、积分与记录更新	`GrowthDiscovery ServiceTest.java`
赵祺	积分兑换逻辑	商品状态、库存与积分余额	`GrowthDiscovery ServiceTest.java`
程泽凯	后台封禁用户	管理权限、目标用户与角色保护	`AdminOpsService Test.java`
程泽凯	举报创建逻辑	原因合法、帖子存在与重复举报	`ReportService Test.java`

成员	代码单元	主要分支	对应测试脚本
程泽凯	通知已读逻辑	通知存在、归属校验与状态更新	`NotificationServiceTest.java`

4.2 各代码单元的测试设计

白盒用例以代码中的判定和提前返回为主线。对于抛出异常的路径，不仅断言发生异常，还核对异常类型并验证 Repository 未发生不应有的保存操作；对于正常路径，则同时检查返回值、实体字段变化和依赖调用。表11给出 12 个单元的路径组与主要断言，路径组描述的是本次设计需要区分的业务出口，不用于替代工具生成的覆盖率报告。

表 **11**　白盒独立路径组与断言目标

代码单元	需要区分的路径组	主要断言
用户注册	账号重复；输入合法并保存；依赖异常传播	异常类型、密码处理、用户保存字段
账号密码登录	用户不存在；密码错误；认证成功	响应状态、令牌或用户信息、失败时无登录态
分页参数修正	页码小于 0；页长越界；参数处于有效范围	修正后的页码、页长和排序参数
帖子搜索入口	关键字为空；分页非法；条件合法	控制器参数传递、返回结构与服务调用
帖子发布	用户不存在；版块不存在；内容非法；保存成功	异常、帖子关联字段、保存与返回对象
评论发布	帖子不存在；用户不存在；内容非法；发布成功	评论关联、通知触发与 Repository 交互
校园身份认证	用户不存在；信息不匹配；首次成功；可信资料保护	认证状态、学院专业字段及失败时不覆盖
每日签到	用户不存在；当天已签到；首次签到	积分增量、签到记录和积分流水
积分兑换	用户异常；商品不可用；库存不足；积分不足；成功	积分库存变化、兑换记录与失败原子性

代码单元	需要区分的路径组	主要断言
后台封禁用户	操作者无权限；目标不存在；目标受保护；操作成功	权限异常、角色保护与封禁状态
举报创建	原因非法；帖子不存在；重复举报；创建成功	举报字段、重复检查和保存次数
通知已读	通知不存在；归属不符；归属正确	资源异常、权限隐藏和已读状态更新

4.2.1 用户、认证与分页单元

用户注册测试首先构造 Repository 已返回同名用户的路径，期望业务层拒绝重复账号且不调用保存；正常路径提供未占用账号和合法字段，检查返回用户、持久化对象及必要字段。账号密码登录位于控制器边界，测试分别模拟用户不存在、密码校验失败和认证成功，避免只覆盖 HTTP 200 的正常分支。分页工具类虽然代码量较小，但被多个列表接口复用，因此负页码和过大页长属于高影响边界；测试需要确认修正结果一致，防止不同控制器各自采用不同分页规则。

4.2.2 帖子搜索、发布与评论单元

帖子搜索入口的重点是控制器是否将关键字与修正后的分页对象准确传递给服务层。空关键字应按接口契约返回默认结果或可接受的空查询结果，而不能形成未处理异常。帖子发布逻辑依赖当前用户和版块，两者任一不存在都应在保存前终止。正常发布路径还需核对作者、版块、标题和正文是否写入目标实体。评论发布除验证帖子与用户外，还需关注评论保存后可能触发的通知行为，避免出现评论已保存但通知关联错误的部分成功状态。

4.2.3 身份认证、签到与兑换单元

校园身份认证测试不仅判断认证状态，还验证失败时不会覆盖用户原有学院、专业和学号。对于已经具有可信认证信息的用户，更新资料时应保护可信字段，防止普通资料编辑破坏认证结果。每日签到测试将“当天首次签到”和“当天重复签到”分开，前者必须同时产生积分变化、签到记录和流水，后者不得重复写入。积分兑换的四项前置条件均可能形成独立失败出口，因此正常路径之外还要分别构造商品不可用、库存不足和积分不足对象，并核对失败后状态保持不变。

4.2.4 后台治理、举报与通知单元

后台封禁用户属于高权限操作，测试顺序先验证操作者权限，再验证目标存在性和目标角色保护。这样可以确认普通用户无法进入管理分支，同时避免管理员误封受保护账号。举报创建逻辑需要限制允许的举报原因，并在保存前确认目标帖子和重复记录；对重复举报的处理应保持幂等，不能生成多条等价记录。通知已读逻辑采用归属校验，通知不存在或属于其他用户时均不得泄露通知内容，只有归属正确时才能更新已读状态。

4.3 控制流与基本路径分析

以“积分兑换”为代表，其业务判断依次涉及用户、商品状态、库存和积分余额。图2将每个提前失败分支单独表示。该控制流包含 4 个判定节点，按 $V(G)=P+1$ 计算得到环形复杂度 5，因此至少需要 5 条独立路径覆盖正常兑换及 4 类失败出口。

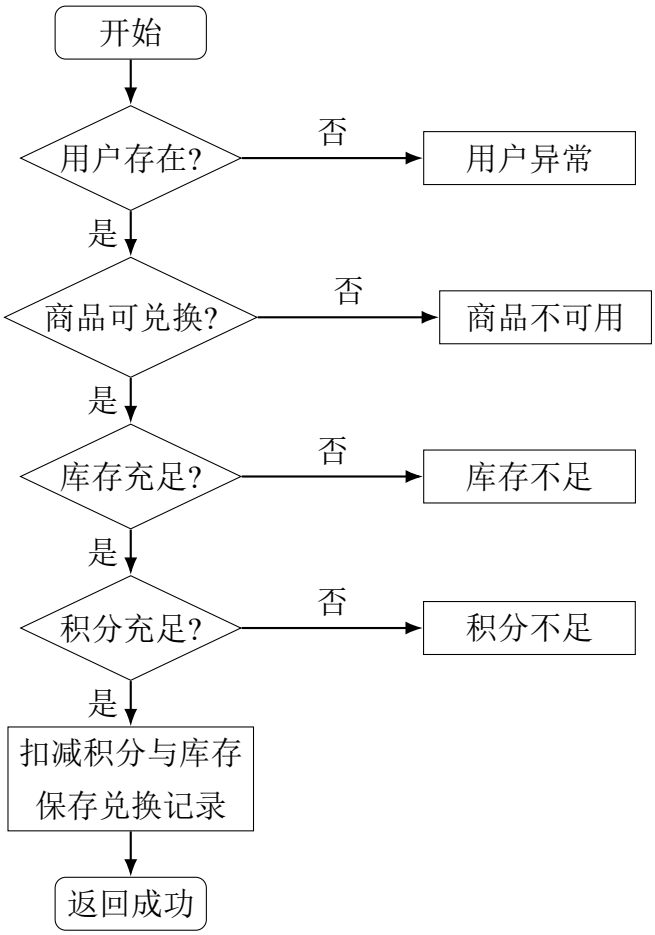


图 **2**　积分兑换逻辑控制流图

对应独立路径为：用户不存在；用户存在但商品不可兑换；商品可兑换但库存不足；库存充足但积分不足；所有条件满足并完成兑换。测试数据分别构造空用户、下架商品、

零库存、余额小于所需积分和合法兑换对象。除返回结果外，测试还校验 Repository 保存次数、积分变化、库存变化及兑换记录字段，从而避免“接口返回成功但状态未正确更新”的假通过。

环形复杂度反映获得基本路径覆盖所需的独立路径数量，但并不等价于业务用例总数。某一独立路径可能还包含多个边界值，例如“积分不足”路径至少可以使用余额为 0 和余额比所需积分少 1 两类数据。相反，一些理论条件组合在执行上不可达，例如用户不存在时无法继续判断其积分余额。设计测试时应保留可达的业务路径，并通过 Mock 对象精确控制每个 Repository 返回值。

在正常兑换路径中，断言顺序按照状态变化展开：首先确认返回结果表示成功，其次确认用户积分减少量等于商品价格，再确认商品库存减少 1，最后检查兑换记录关联了正确用户和商品。失败路径则采用相反检查，重点验证积分、库存和兑换记录均未改变。通过同时验证正向结果和副作用，可以发现仅返回值正确但内部状态写入不完整的问题。

4.4 条件组合分析

对于通知已读逻辑，条件包括“通知是否存在”和“通知是否属于当前用户”。两项条件形成四种理论组合，但当通知不存在时无法继续检查归属，因此实际可执行组合为：通知不存在；通知存在但不属于当前用户；通知存在且属于当前用户。测试分别期望资源不存在异常、权限隐藏异常和已读状态更新。对于后台封禁用户逻辑，还需组合管理员身份、目标账号存在性和目标角色，确保普通用户不能调用管理操作，管理员不能误封受保护角色，合法目标才能更新封禁状态。

条件–判定覆盖关注每个整体判定的真、假结果以及原子条件的取值。例如后台封禁逻辑中的“操作者是否为管理员”和“目标是否为受保护角色”均需要至少出现一次真值与假值。条件组合覆盖进一步检查不同条件同时出现时的行为，但应排除业务上不可达的组合。测试报告因此同时记录输入对象的角色、Repository 返回值和期望出口，使每条用例都能对应到具体判断，而不是仅以方法名推测覆盖情况。

Mockito 交互验证用于补充分支断言。遇到用户不存在、商品不可用或权限不足时，测试使用 `verify(..., never())` 确认保存方法未被调用；正常路径则检查保存次数和参数对象。该做法能够识别“已经抛出异常但错误数据仍被部分写入”的缺陷。对于只读控制器，还需要验证服务方法接收到的分页和关键字参数，确保测试覆盖控制层与服务层之间的契约。

4.5 自动化执行结果

在 `jiaodaluntan/backend` 目录执行 `mvn test`，Surefire 报告记录 18 个测试类、85 项测试，失败 0、错误 0、跳过 0。四份个人报告引用的是同一次完整后端回归结果，因

此综合报告仅统计一次，不将相同的 85 项测试重复相加。当前执行未启用 JaCoCo，故不报告语句覆盖率、分支覆盖率或行覆盖率百分比。

表 **12**　**JUnit** 执行结果

指标	数量	指标	数量
测试类	18	测试项	85
失败	0	错误	0
跳过	0	通过率	100%

测试结果说明现有输入组合下的服务层与控制层逻辑符合断言，但 100% 通过不等同于不存在缺陷。剩余风险主要来自未纳入单元测试的数据库并发、事务回滚、真实 JWT 过滤链和前后端字段契约。后续应在 CI 中启用 JaCoCo，并对高风险方法设定分支覆盖率门槛，同时保留本轮测试作为回归基线。

五、 练习 3：性能测试

5.1 场景设计与统计口径

性能测试脚本采用固定并发工作线程持续请求公共接口。负载测试模拟常规混合浏览；压力测试将并发数从 8 逐级提高至 48；容量测试保持并发数 12 并改变分页大小；稳定性测试以并发数 10 持续 20 秒混合访问。每个请求记录耗时与状态，场景结束后计算总请求数、成功数、错误率、吞吐量、平均值、P50、P90、P95、P99 和最大响应时间。

5.2 性能指标定义

吞吐量表示单位时间内完成的请求数，计算式为 $Q = N/T$，其中 N 为场景完成请求总数，T 为实际持续时间。错误率按 $E = N_{error}/N \times 100\%$ 计算，连接失败、超时或不符合成功条件的响应均应计入错误。平均响应时间用于描述总体处理水平，但容易受少量极慢请求影响，因此报告同时使用 P95 和 P99 观察长尾。P95 表示 95% 的请求响应时间不超过该值，剩余 5% 请求可能更慢。

性能测试结果必须结合场景解释。吞吐量较高并不必然表示体验更好，如果并发增加后吞吐量保持不变而 P95 持续升高，说明系统可能已经接近当前环境下的饱和区间。错误率为 0 也不能单独证明系统不存在性能问题，因为短时测试可能尚未触发连接池耗尽、内存泄漏或日志增长。因此本报告同时比较不同并发和不同分页大小的相对变化，并在结论中明确时间和环境限制。

表 **13**　性能场景配置与验证目标

场景	并发配置	持续时间	请求组合	验证目标
常规负载	8	约 12 秒	列表、热门、搜索、版块	观察混合浏览基线
阶梯压力	8/16/32/48	每级约 6 秒	列表与搜索	比较并发增长下吞吐量和长尾延迟
分页容量	12	每组约 5 秒	列表与搜索，页长 10/20/50/100	观察响应数据量变化的影响
短时稳定性	10	约 20 秒	列表、热门、搜索、版块	检查持续重复访问是否出现错误或退化

5.3 原始结果汇总

表14列出原始 JSON 中的 10 组场景结果。所有场景均未记录请求错误，因此性能差异主要体现在吞吐量和响应时间分位数。

表 **14**　性能测试原始结果

场景	并发	请求数	吞吐量/req·s^{-1}	平均/ms	P95/ms	P99/ms
LOAD-01 混合浏览	8	13768	1147.24	6.96	15.78	26.96
STRESS-8	8	14338	2388.87	3.34	5.83	8.22
STRESS-16	16	15591	2597.63	6.15	9.50	13.10
STRESS-32	32	14775	2461.27	12.99	21.27	27.75
STRESS-48	48	15561	2590.05	18.51	27.59	37.60
CAP-10	12	17077	3414.72	3.51	5.05	7.55
CAP-20	12	15355	3069.77	3.90	5.94	7.94
CAP-50	12	11960	2390.57	5.01	7.87	9.88
CAP-100	12	11696	2337.80	5.13	7.95	10.61
STAB-01 持续访问	10	60825	3041.10	3.29	5.34	7.18

5.4 负载测试分析

常规混合浏览场景使用 8 个并发工作线程，在约 12 秒内完成 13768 次请求，吞吐量为 1147.24 req/s，平均响应时间 6.96 ms，P95 为 15.78 ms，P99 为 26.96 ms，错误率

为 0。该场景同时访问列表、热门、搜索和版块接口，其吞吐量低于只包含列表与搜索的压力场景，说明不同接口组合的处理成本并不完全一致。混合负载更接近普通用户在首页切换内容的行为，因此可作为后续版本对比的基线。

最大响应时间为 194.95 ms，明显高于 P99，但单次最大值不能直接代表稳定性能。由于原始数据只包含本轮请求耗时，没有多轮重复试验和服务器资源曲线，无法判断该最大值来自 JIT 预热、首次缓存、操作系统调度或接口本身。后续基线测试应增加预热阶段，并至少重复三轮，分别报告均值、标准差和置信区间，从而区分偶然波动与稳定回退。

5.5 压力测试趋势

图3采用柱线组合方式展示并发阶梯下的吞吐量与 P95 响应时间。吞吐量在 16 并发时达到 2597.63 req/s，随后在 32 和 48 并发下维持在 2461.27 至 2590.05 req/s；P95 则随并发提高而持续上升，从 5.83 ms 增加到 27.59 ms。该现象说明系统在本机短时测试中仍能保持零错误，但增加并发主要带来排队等待，吞吐量未继续线性增长。

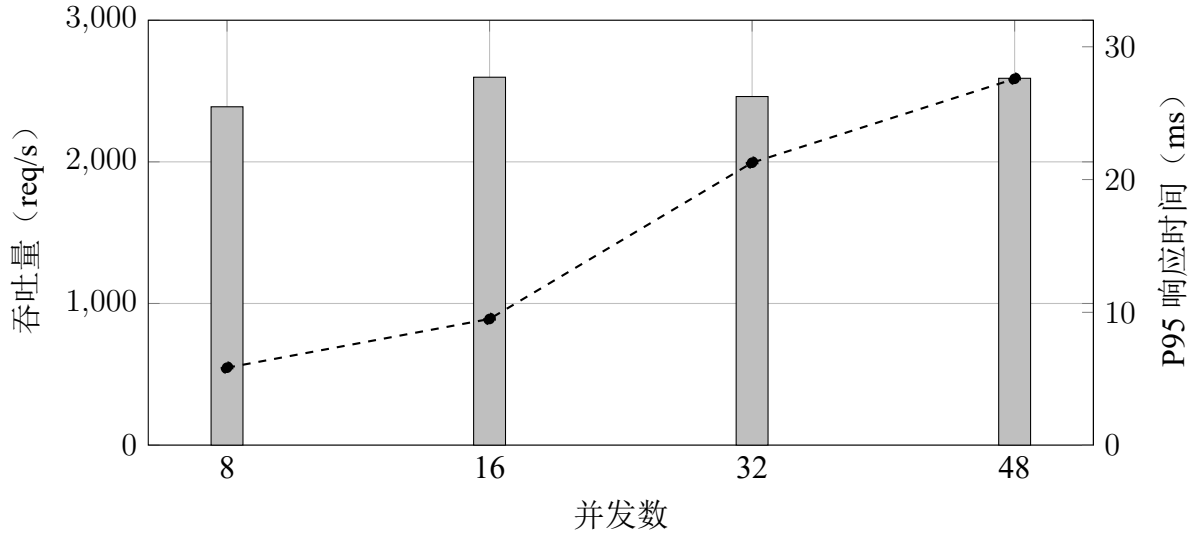


图 **3**　并发阶梯下吞吐量与 **P95** 响应时间

从相邻阶梯观察，8 并发提升到 16 并发时，吞吐量增加约 8.74%，P95 从 5.83 ms 增加到 9.50 ms；并发增加到 32 时，吞吐量反而下降约 5.25%，而 P95 上升到 21.27 ms；48 并发下吞吐量恢复到 2590.05 req/s，但 P95 继续上升到 27.59 ms。由此可见，16 并发以后新增并发主要增加等待时间，没有形成稳定的吞吐量增长。

本轮没有出现错误或超时，因此无法从错误率确定明确崩溃点。更合理的解释是，32 至 48 并发已经表现出延迟增长快于吞吐量增长的饱和趋势。后续压力测试应继续提高并发，直到错误率、超时率或吞吐量明显恶化，同时采集 CPU 利用率、活动线程数和数据库连接池等待时间，才能判断限制来自应用线程、数据库查询还是本机网络栈。

5.6 容量与稳定性分析

容量测试中，分页大小从 10 增加到 100 时，吞吐量从 3414.72 req/s 下降到 2337.80 req/s，P95 从 5.05 ms 上升到 7.95 ms。分页 20 至 50 之间的吞吐量下降较为明显，说明返回记录数和序列化数据量会占用更多处理时间。实际接口应限制过大的 `size` 参数，并结合数据库索引、投影查询和响应压缩降低大页请求成本。

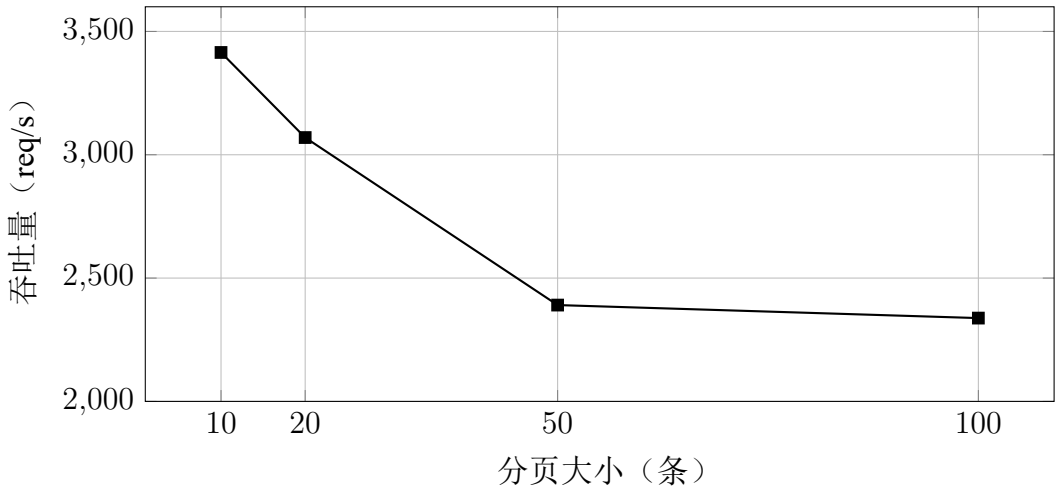


图 **4**　分页大小与吞吐量关系

以分页 10 为基准，分页 20 的吞吐量下降约 10.10%，分页 50 下降约 30.00%，分页 100 下降约 31.54%。分页 50 与 100 的吞吐量接近，说明在当前数据集和接口实现下，固定查询开销与返回数据处理共同影响结果，不能简单认定响应条数翻倍必然使耗时翻倍。P99 则从 7.55 ms 逐步上升到 10.61 ms，表明大页请求对尾部响应仍有持续影响。

容量测试使用的是分页大小而非数据库记录总量，因此它评价的是单次响应容量，不是系统可保存的最大帖子数量。若要评估数据库容量，需要构造不同规模的数据集，在相同查询条件下比较索引命中、SQL 执行计划、响应体大小和内存占用。本轮结果只能支持“限制过大分页能够降低单次请求成本”这一结论，不能据此推断数据库的极限记录数。

稳定性场景持续 20 秒，共完成 60825 次请求，吞吐量为 3041.10 req/s，平均响应时间 3.29 ms，P99 为 7.18 ms，错误率为 0。该结果可以证明系统未在短时重复访问中立即出现连接异常或持续退化，但 20 秒不足以评价内存泄漏、连接池耗尽、日志膨胀和长期缓存变化。正式上线前仍应扩展为 30 分钟以上的持续测试，并同步采集 CPU、内存、GC、数据库连接池和磁盘 I/O 指标。

稳定性测试还应按时间窗口拆分结果，例如每分钟计算一次吞吐量和 P95，再观察曲线是否持续上升或下降。当前原始文件提供的是场景整体统计，无法判断第 1 秒与第 20 秒之间是否存在轻微趋势。后续脚本可增加分桶统计和资源采样，使“持续稳定”由时间序列证据支持，而不是仅由场景结束时的总体平均值支持。

5.7 性能结论的适用范围

本轮测试以本机回环地址和公共读接口为主，网络传输距离短，且没有覆盖登录、写数据库、图片上传和复杂后台操作。因此，结果适合用于比较不同并发和分页配置的相对趋势，不应直接作为公网生产环境的服务等级承诺。后续测试需引入独立压力机、稳定数据集和服务器监控，并将不同轮次的均值与标准差同时报告，以提高统计可信度。

测试期间所有场景错误率均为 0，这是对当前接口可用性的正面证据，但也说明本轮尚未找到系统失效边界。公共读接口通常比发帖、评论、签到和兑换等写操作开销更低，后者还涉及事务、锁竞争和多表更新。下一阶段应将已登录读写混合场景纳入测试，并使用独立测试账号清理数据，避免大量写入污染课程演示环境。

性能基线应与环境配置绑定。只有在 JDK 版本、数据库数据量、应用配置、压力机和网络条件一致时，两个版本的吞吐量与分位数才具有直接可比性。若环境发生变化，应重新建立基线并说明差异。本报告保留原始 JSON 和脚本，目的正是使后续人员能够复现同一场景，而不是只保留截图中的最终数字。

六、 综合质量评价与改进建议

6.1 测试结论

三个练习形成了由外到内的测试链。黑盒测试验证了 15 个用户功能点并发现 6 项实际缺陷，说明页面行为和跨模块状态仍是当前主要问题来源；白盒测试的 85 项自动化用例全部通过，表明关键服务逻辑在已设计条件下保持稳定；性能测试在 10 组短时场景中均未产生请求错误，公共查询接口具有较好的本机处理能力。综合判断，系统已经具备课程设计演示与验收所需的核心可用性，但尚未达到可以忽略风险的生产级成熟度。

6.2 缺陷优先级与回归策略

已修复的通知面板、兑换记录和旧数据库启动问题均可能阻断用户流程，应长期保留对应自动化回归。待修复的中文乱码和积分展示不一致影响信息可信度，优先级高于单纯视觉优化；身份认证覆盖信息前缺少确认属于低频但不可逆风险，应增加确认对话框和变更日志。每次后端发布至少执行 `mvn test`，每次前端交互调整应回归通知、兑换、签到和资料编辑，数据库结构变化则必须增加旧数据迁移测试。

表 **15**　缺陷原因分析与回归重点

编号	可能原因类别	修复或控制措施	回归重点
BUG-001	前端显示状态与 DOM 状态不一致	统一面板显隐状态，补充 `hidden`、`display` 和 `aria-expanded` 检查	桌面与移动宽度下点击、关闭、重复打开
BUG-002	兑换记录序列化或查询契约异常	修正接口返回与关联数据加载，保留 500 响应回归	兑换前后刷新、空记录和多条记录加载
BUG-003	数据库结构升级兼容性不足	为新增非空字段提供迁移默认值或分阶段迁移	使用旧 H2 数据文件启动并完成升级
BUG-004	文件或响应字符编码不统一	统一源码、构建产物和 HTTP 响应为 UTF-8	标题、按钮、详情正文及特殊中文字符
BUG-005	签到记录与积分流水来源不一致	明确单一统计口径并在事务内同步写入	首次签到、重复签到、刷新与历史记录
BUG-006	高影响资料变更缺少交互确认	增加二次确认、变更摘要和必要审计日志	取消确认、确认覆盖及刷新后的资料状态

缺陷严重程度根据用户影响、发生频率和恢复成本综合确定。通知不可见、兑换记录 500 和旧数据库无法启动会直接阻断主要流程，因此被列为高严重程度；中文乱码和积分展示不一致虽然不一定阻止操作，但会降低信息可信度，列为中等严重程度；身份资料覆盖缺少确认属于可用性与误操作风险，当前列为低严重程度。若后续发现覆盖行为导致不可恢复的数据错误，应重新提高其优先级。

6.3 自动化与持续集成建议

现有 Selenium、JUnit 和 Node.js 脚本已经具备基础自动化能力，但三类脚本尚未形成统一流水线。建议在 CI 中按“单元测试–应用启动–接口健康检查–关键 Selenium 场景–短时性能基线”顺序执行，并归档 Surefire XML、截图、性能 JSON 和失败日志。对单元测试启用 JaCoCo 后，可针对服务层建立分支覆盖率基线；对性能测试则应保留历史结果，通过同机同配置对比识别明显回退。

七、 实验总结

综合测试表明，测试方法的选择应与问题层次相匹配。黑盒方法适合从需求和用户视角发现流程中断，白盒方法适合验证复杂条件和异常出口，性能测试则用于观察负载变化引起的响应趋势。仅依赖某一类测试会形成明显盲区：页面成功不能证明内部状态正确，单元测试通过不能证明前端可用，短时零错误也不能证明长期稳定。

本次工作进一步说明，真实测试报告的核心是可追溯性。用例必须能够关联需求，结论必须能够关联执行记录，缺陷必须能够关联截图或日志，修复后还应给出回归证据。通过统一整理三个练习，原先分散的脚本、数据和报告被组织为一条完整质量证据链，为后续版本迭代提供了可重复执行的基线。

A 自动化脚本清单

本附录收录三类测试实际使用的脚本。黑盒测试脚本位于 `scripts/blackbox`，白盒 JUnit 脚本位于 `scripts/unit`，性能测试脚本与原始结果位于 `scripts/performance`。完整文件与本报告一同保存，以下代码直接从对应文件读取，避免报告代码与实际脚本不一致。

1.1 Selenium 黑盒测试脚本

```
package com.swjtu.forum;

import org.junit.jupiter.api.*;
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.Select;
import org.openqa.selenium.support.ui.WebDriverWait;

import java.time.Duration;

import static org.junit.jupiter.api.Assertions.*;

@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
public class ForumBlackBoxTest {
    private static final String BASE_URL = System.getProperty("forum.baseUrl",
    ↪  "http://localhost:4173");
    private static WebDriver driver;
    private static WebDriverWait wait;
    private static final String ADMIN = "admin";
    private static final String ADMIN_PASSWORD = "admin123456";

    @BeforeAll
    static void setUp() {
        ChromeOptions options = new ChromeOptions();
        options.addArguments("--remote-allow-origins=*", "--window-size=1440,1000");
        options.setBinary("C:\\Program Files\\Google\\Chrome\\Application\\chrome.exe");
        driver = new ChromeDriver(options);
        wait = new WebDriverWait(driver, Duration.ofSeconds(15));
    }

    @AfterAll
    static void tearDown() {
        if (driver != null) {
            driver.quit();
        }
```

```
}

@Test
@Order(1)
void loginBrowseFilterAndSearch() {
    loginAsAdmin();
    driver.get(BASE_URL + "/home_recommended_desktop/code.html");
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("main
    ↪  article")));
    assertTrue(driver.findElements(By.cssSelector("main article")).size() >= 1);

    Select section = new Select(driver.findElement(By.cssSelector("main select")));
    section.selectByIndex(1);
    click(By.cssSelector("[data-search-form] button[type='submit']"));
    wait.until(ExpectedConditions.urlContains("sectionId="));

    driver.get(BASE_URL + "/home_recommended_desktop/code.html");
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("[data-search」
    ↪  -form] input[name='keyword']")));
    driver.findElement(By.cssSelector("[data-search-form]
    ↪  input[name='keyword']")).sendKeys("签到");
    click(By.cssSelector("[data-search-form] button[type='submit']"));
    wait.until(ExpectedConditions.or(
            ExpectedConditions.urlContains("keyword="),
            ExpectedConditions.textToBePresentInElementLocated(By.cssSelector("body"),
            ↪  "签到")
    ));
}

@Test
@Order(2)
void notificationPanelMustBecomeVisible() {
    loginAsAdmin();
    driver.get(BASE_URL + "/home_recommended_desktop/code.html");
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("[data-notif-」
    ↪  toggle]")));
    click(By.cssSelector("[data-notif-toggle]"));
    WebElement panel = wait.until(ExpectedConditions.presenceOfElementLocated(By.cssSelec」
    ↪  tor("[data-notif-panel]")));
    Boolean hidden = (Boolean) ((JavascriptExecutor) driver).executeScript("return
    ↪  arguments[0].hidden;", panel);
    String display = (String) ((JavascriptExecutor) driver).executeScript("return
    ↪  getComputedStyle(arguments[0]).display;", panel);
    assertFalse(hidden, "notification panel should not be hidden after clicking bell");
    assertNotEquals("none", display, "notification panel display should not be none");
    assertEquals("true", driver.findElement(By.cssSelector("[data-notif-toggle]")).getAtt」
    ↪  ribute("aria-expanded"));
```

```
}

@Test
@Order(3)
void rewardsAndRedemptionsMustLoadWithoutServerError() {
    loginAsAdmin();
    driver.get(BASE_URL + "/rewards_mall/code.html");
    wait.until(ExpectedConditions.not(ExpectedConditions.textToBePresentInElementLocated(」
    ↪  By.cssSelector("body"), "页面加载失败")));
    assertFalse(driver.getPageSource().contains("Internal Server Error"));
    assertTrue(driver.getPageSource().contains("积分") ||
    ↪  driver.findElements(By.cssSelector("[data-redeem]")).size() > 0);
}

@Test
@Order(4)
void createCommentLikeFavoriteAndFollow() {
    loginAsAdmin();
    driver.get(BASE_URL + "/post_details/code.html?id=1");
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("body")));

    if (driver.findElements(By.cssSelector("[data-comment-form]
    ↪  textarea[name='content']")).size() > 0) {
        driver.findElement(By.cssSelector("[data-comment-form]
        ↪  textarea[name='content']")).sendKeys("Selenium black-box comment");
        click(By.cssSelector("[data-comment-form] button[type='submit']"));
        wait.until(ExpectedConditions.textToBePresentInElementLocated(By.cssSelector("bod」
        ↪  y"), "Selenium black-box comment"));
    }
    if (driver.findElements(By.cssSelector("[data-like-post]")).size() > 0) {
        click(By.cssSelector("[data-like-post]"));
    }
    if (driver.findElements(By.cssSelector("[data-favorite]")).size() > 0) {
        click(By.cssSelector("[data-favorite]"));
    }
    if (driver.findElements(By.cssSelector("[data-follow]")).size() > 0) {
        click(By.cssSelector("[data-follow]"));
        wait.until(ExpectedConditions.presenceOfElementLocated(By.cssSelector("[data-foll」
        ↪  ow]")));
    }
}

private static void loginAsAdmin() {
    driver.get(BASE_URL + "/home_recommended_desktop/code.html");
    try {
        new WebDriverWait(driver, Duration.ofSeconds(4))
```

```
                .until(ExpectedConditions.presenceOfElementLocated(By.cssSelector("[data-
                ↪  notif-toggle]")));
            return;
        } catch (TimeoutException ignored) {
            ((JavascriptExecutor) driver).executeScript("window.localStorage.clear();
            ↪  window.sessionStorage.clear();");
        }

        driver.get(BASE_URL + "/login_register/code.html");
        wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("[data-auth-f
        ↪  orm='login']")));
        WebElement username = driver.findElement(By.cssSelector("[data-auth-form='login']
        ↪  input[name='username']"));
        username.clear();
        username.sendKeys(ADMIN);
        WebElement password = driver.findElement(By.cssSelector("[data-auth-form='login']
        ↪  input[name='password']"));
        password.clear();
        password.sendKeys(ADMIN_PASSWORD);
        click(By.cssSelector("[data-auth-form='login'] [data-auth-submit]"));
        wait.until(ExpectedConditions.or(
                ExpectedConditions.urlContains("home_recommended"),
                ExpectedConditions.presenceOfElementLocated(By.cssSelector("[data-notif-toggl
                ↪  e]"))
        ));
    }

    private static void click(By locator) {
        StaleElementReferenceException last = null;
        for (int i = 0; i < 3; i++) {
            try {
                WebElement element = wait.until(ExpectedConditions.refreshed(
                        ExpectedConditions.presenceOfElementLocated(locator)));
                ((JavascriptExecutor)
                ↪  driver).executeScript("arguments[0].scrollIntoView({block:'center'});",
                ↪  element);
                ((JavascriptExecutor) driver).executeScript("arguments[0].click();", element);
                return;
            } catch (StaleElementReferenceException error) {
                last = error;
                try {
                    Thread.sleep(300);
                } catch (InterruptedException interrupted) {
                    Thread.currentThread().interrupt();
                }
            }
        }
```

```
        throw last;
    }
}
```

B JUnit 白盒测试脚本

2.1 用户服务测试

```
package com.swjtu.forum.service;

import com.swjtu.forum.dto.CampusIdentityVerificationResult;
import com.swjtu.forum.dto.CampusIdentityVerifyRequest;
import com.swjtu.forum.dto.LoginRequest;
import com.swjtu.forum.dto.RegisterRequest;
import com.swjtu.forum.dto.UserProfileUpdateRequest;
import com.swjtu.forum.entity.Tag;
import com.swjtu.forum.entity.User;
import com.swjtu.forum.repository.TagRepository;
import com.swjtu.forum.repository.UserRepository;
import com.swjtu.forum.security.JwtUtil;
import com.swjtu.forum.security.LoginAttemptService;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.ArgumentCaptor;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.security.crypto.password.PasswordEncoder;

import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;

import static org.assertj.core.api.Assertions.assertThat;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock
    private UserRepository userRepository;
```

```
@Mock
private PasswordEncoder passwordEncoder;

@Mock
private TagRepository tagRepository;

private UserService userService;
private StubJwtUtil jwtUtil;
private User user;

@BeforeEach
void setUp() {
    jwtUtil = new StubJwtUtil();
    userService = new UserService(userRepository, passwordEncoder, jwtUtil, new
    ↪  LoginAttemptService(), null, tagRepository);

    user = new User();
    user.setId(1L);
    user.setUsername("alice");
    user.setPassword("encoded-password");
    user.setEmail("alice@example.com");
    user.setEnabled(true);
}

@Test
void registerReturnsDirectActivationMetadataAndEnabledUser() {
    RegisterRequest request = new RegisterRequest();
    request.setUsername("alice");
    request.setPassword("password");
    request.setEmail("alice@example.com");
    request.setStudentId("20240001");
    request.setCollege("Computer");
    request.setMajor("Software");

    when(userRepository.existsByUsername("alice")).thenReturn(false);
    when(userRepository.existsByEmail("alice@example.com")).thenReturn(false);
    when(passwordEncoder.encode("password")).thenReturn("encoded-password");
    when(userRepository.save(any(User.class))).thenAnswer(invocation ->
    ↪  invocation.getArgument(0));

    Map<String, Object> result = userService.register(request);

    assertThat(result).containsEntry("token", "stub-token-alice");
    assertThat(result).containsEntry("activationMode", "direct");
    assertThat(result).containsEntry("activationRequired", false);
    assertThat(((User) result.get("user")).getEnabled()).isTrue();
}
```

```
@Test
void loginReturnsDirectActivationMetadata() {
    LoginRequest request = new LoginRequest();
    request.setUsername("alice");
    request.setPassword("password");

    when(userRepository.findByUsername("alice")).thenReturn(Optional.of(user));
    when(passwordEncoder.matches("password", "encoded-password")).thenReturn(true);

    Map<String, Object> result = userService.login(request);

    assertThat(result).containsEntry("token", "stub-token-alice");
    assertThat(result).containsEntry("activationMode", "direct");
    assertThat(result).containsEntry("activationRequired", false);
}

@Test
void updateCurrentUserProfilePersistsPhaseOneFields() {
    UserProfileUpdateRequest request = new UserProfileUpdateRequest();
    request.setAvatar("https://example.com/avatar.png");
    request.setBio("bio");
    request.setCollege("Computer");
    request.setMajor("Software");
    request.setStudentId("20240001");

    when(userRepository.findByUsername("alice")).thenReturn(Optional.of(user));
    when(userRepository.save(any(User.class))).thenAnswer(invocation ->
    ↪  invocation.getArgument(0));

    User updated = userService.updateCurrentUserProfile("alice", request);

    assertThat(updated.getAvatar()).isEqualTo("https://example.com/avatar.png");
    assertThat(updated.getBio()).isEqualTo("bio");
    assertThat(updated.getCollege()).isEqualTo("Computer");
    assertThat(updated.getMajor()).isEqualTo("Software");
    assertThat(updated.getStudentId()).isEqualTo("20240001");

    ArgumentCaptor<User> savedUser = ArgumentCaptor.forClass(User.class);
    verify(userRepository).save(savedUser.capture());
    assertThat(savedUser.getValue().getCollege()).isEqualTo("Computer");
}

@Test
void verifyCampusIdentitySyncsTrustedProfileFieldsOnSuccess() {
    CampusIdentityVerifyRequest request = new CampusIdentityVerifyRequest();
    request.setStudentId("20240001");
```

```
    request.setCampusEmail("20240001@swjtu.edu.cn");

    when(userRepository.findByUsername("alice")).thenReturn(Optional.of(user));
    when(userRepository.save(any(User.class))).thenAnswer(invocation ->
    ↪  invocation.getArgument(0));

    CampusIdentityVerificationResult result = userService.verifyCampusIdentity("alice",
    ↪  request);

    assertThat(result.isVerified()).isTrue();
    assertThat(result.getStatus()).isEqualTo("VERIFIED");
    assertThat(result.getUser().getCollege()).isEqualTo(" 计算机与人工智能学院");
    assertThat(result.getUser().getMajor()).isEqualTo(" 软件工程");
    assertThat(result.getUser().getCampusVerified()).isTrue();
    assertThat(result.getUser().getCampusEmail()).isEqualTo("20240001@swjtu.edu.cn");
}

@Test
void verifyCampusIdentityReturnsFailureWithoutOverwritingProfileOnMismatch() {
    user.setCollege("Original College");
    user.setMajor("Original Major");
    user.setStudentId("legacy");

    CampusIdentityVerifyRequest request = new CampusIdentityVerifyRequest();
    request.setStudentId("20240001");
    request.setCampusEmail("wrong@swjtu.edu.cn");

    when(userRepository.findByUsername("alice")).thenReturn(Optional.of(user));
    when(userRepository.save(any(User.class))).thenAnswer(invocation ->
    ↪  invocation.getArgument(0));

    CampusIdentityVerificationResult result = userService.verifyCampusIdentity("alice",
    ↪  request);

    assertThat(result.isVerified()).isFalse();
    assertThat(result.getStatus()).isEqualTo("FAILED");
    assertThat(result.getMessage()).isEqualTo(" 校园邮箱与学号不匹配");
    assertThat(result.getUser().getCollege()).isEqualTo("Original College");
    assertThat(result.getUser().getMajor()).isEqualTo("Original Major");
    assertThat(result.getUser().getStudentId()).isEqualTo("legacy");
}

@Test
void updateCurrentUserProfileDoesNotOverrideTrustedCampusFieldsAfterVerification() {
    user.setCampusVerified(true);
    user.setCollege(" 计算机与人工智能学院");
    user.setMajor(" 软件工程");
```

```
    user.setStudentId("20240001");

    UserProfileUpdateRequest request = new UserProfileUpdateRequest();
    request.setAvatar("https://example.com/avatar.png");
    request.setBio("bio");
    request.setCollege("Other College");
    request.setMajor("Other Major");
    request.setStudentId("99999999");

    when(userRepository.findByUsername("alice")).thenReturn(Optional.of(user));
    when(userRepository.save(any(User.class))).thenAnswer(invocation ->
    ↪  invocation.getArgument(0));

    User updated = userService.updateCurrentUserProfile("alice", request);

    assertThat(updated.getAvatar()).isEqualTo("https://example.com/avatar.png");
    assertThat(updated.getBio()).isEqualTo("bio");
    assertThat(updated.getCollege()).isEqualTo(" 计算机与人工智能学院");
    assertThat(updated.getMajor()).isEqualTo(" 软件工程");
    assertThat(updated.getStudentId()).isEqualTo("20240001");
}

@Test
void updateCurrentUserTagsReplacesOnlyInterestTags() {
    Tag postTag = tag(1L, "Spring Boot", "post");
    Tag oldInterest = tag(2L, " 考研保研", "interest");
    Tag newInterest = tag(3L, " 课程互助", "interest");
    user.setTags(new LinkedHashSet<>(List.of(postTag, oldInterest)));

    when(userRepository.findByUsername("alice")).thenReturn(Optional.of(user));
    when(tagRepository.findAllById(Set.of(3L))).thenReturn(List.of(newInterest));
    when(userRepository.save(any(User.class))).thenAnswer(invocation ->
    ↪  invocation.getArgument(0));

    Set<Tag> selected = userService.updateCurrentUserTags("alice", List.of(3L));

    assertThat(selected).containsExactly(newInterest);
    assertThat(user.getTags()).containsExactly(postTag, newInterest);
    verify(userRepository).save(user);
}

private Tag tag(Long id, String name, String category) {
    Tag tag = new Tag();
    tag.setId(id);
    tag.setName(name);
    tag.setCategory(category);
    return tag;
```

```
    }

    private static class StubJwtUtil extends JwtUtil {
        @Override
        public String generateToken(String username) {
            return "stub-token-" + username;
        }
    }
}
```

2.2 认证控制器测试

```
package com.swjtu.forum.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.swjtu.forum.dto.LoginRequest;
import com.swjtu.forum.dto.RegisterRequest;
import com.swjtu.forum.dto.ApiResponse;
import com.swjtu.forum.entity.User;
import com.swjtu.forum.exception.GlobalExceptionHandler;
import com.swjtu.forum.service.UserService;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.setup.MockMvcBuilders;
import org.springframework.validation.beanvalidation.LocalValidatorFactoryBean;

import java.util.Map;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@ExtendWith(MockitoExtension.class)
class AuthControllerTest {

    private MockMvc mockMvc;
    private final ObjectMapper objectMapper = new ObjectMapper();

    @Mock
    private UserService userService;
```

```
@BeforeEach
void setUp() {
    LocalValidatorFactoryBean validator = new LocalValidatorFactoryBean();
    validator.afterPropertiesSet();
    AuthController controller = new AuthController(userService);
    mockMvc = MockMvcBuilders.standaloneSetup(controller)
            .setControllerAdvice(new GlobalExceptionHandler())
            .setValidator(validator)
            .build();
}

@Test
void registerSuccess() throws Exception {
    RegisterRequest request = new RegisterRequest();
    request.setUsername("testuser");
    request.setPassword("password123");
    request.setEmail("test@swjtu.edu.cn");

    when(userService.register(any(RegisterRequest.class)))
            .thenReturn(Map.of("token", "jwt-token", "username", "testuser"));

    mockMvc.perform(post("/auth/register")
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.code").value(200))
            .andExpect(jsonPath("$.message").value(" 注册成功"));
}

@Test
void registerFailsWithBlankUsername() throws Exception {
    RegisterRequest request = new RegisterRequest();
    request.setUsername("");
    request.setPassword("password123");
    request.setEmail("test@swjtu.edu.cn");

    mockMvc.perform(post("/auth/register")
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isBadRequest());
}

@Test
void registerFailsWithShortPassword() throws Exception {
    RegisterRequest request = new RegisterRequest();
    request.setUsername("testuser");
```

```
    request.setPassword("123");
    request.setEmail("test@swjtu.edu.cn");

    mockMvc.perform(post("/auth/register")
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isBadRequest());
}

@Test
void loginSuccess() throws Exception {
    LoginRequest request = new LoginRequest();
    request.setUsername("testuser");
    request.setPassword("password123");

    when(userService.login(any(LoginRequest.class)))
            .thenReturn(Map.of("token", "jwt-token", "username", "testuser"));

    mockMvc.perform(post("/auth/login")
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.code").value(200))
            .andExpect(jsonPath("$.message").value(" 登录成功"));
}

@Test
void loginFailsWithBlankPassword() throws Exception {
    LoginRequest request = new LoginRequest();
    request.setUsername("testuser");
    request.setPassword("");

    mockMvc.perform(post("/auth/login")
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isBadRequest());
}

@Test
void loginFailsWithBlankUsername() throws Exception {
    LoginRequest request = new LoginRequest();
    request.setUsername("");
    request.setPassword("password123");

    mockMvc.perform(post("/auth/login")
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(objectMapper.writeValueAsString(request)))
```

```
                .andExpect(status().isBadRequest());
    }
}
```

2.3 分页工具测试

```
package com.swjtu.forum.util;

import org.junit.jupiter.api.Test;
import org.springframework.data.domain.Pageable;

import static org.junit.jupiter.api.Assertions.assertEquals;

class PageablesTest {

    @Test
    void ofClampsNegativePageAndOversizedRequest() {
        Pageable pageable = Pageables.of(-3, 500);

        assertEquals(0, pageable.getPageNumber());
        assertEquals(Pageables.MAX_PAGE_SIZE, pageable.getPageSize());
    }

    @Test
    void ofUsesDefaultSizeForInvalidSize() {
        Pageable pageable = Pageables.of(2, 0);

        assertEquals(2, pageable.getPageNumber());
        assertEquals(Pageables.DEFAULT_PAGE_SIZE, pageable.getPageSize());
    }
}
```

2.4 成长与发现服务测试

```
package com.swjtu.forum.service;

import com.swjtu.forum.dto.RecommendationConfigRequest;
import com.swjtu.forum.entity.DailyCheckIn;
import com.swjtu.forum.entity.Post;
import com.swjtu.forum.entity.PointsHistory;
import com.swjtu.forum.entity.RecommendationConfig;
import com.swjtu.forum.entity.RedemptionRequest;
import com.swjtu.forum.entity.RewardItem;
import com.swjtu.forum.entity.Section;
import com.swjtu.forum.entity.Tag;
import com.swjtu.forum.entity.User;
```

```
import com.swjtu.forum.exception.ApiException;
import com.swjtu.forum.repository.CommentRepository;
import com.swjtu.forum.repository.DailyCheckInRepository;
import com.swjtu.forum.repository.PointsHistoryRepository;
import com.swjtu.forum.repository.PostDisinterestRepository;
import com.swjtu.forum.repository.PostRepository;
import com.swjtu.forum.repository.RecommendationConfigRepository;
import com.swjtu.forum.repository.RedemptionRequestRepository;
import com.swjtu.forum.repository.RewardItemRepository;
import com.swjtu.forum.repository.TagRepository;
import com.swjtu.forum.repository.UserRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.ArgumentCaptor;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.data.domain.PageImpl;
import org.springframework.data.domain.PageRequest;
import org.springframework.dao.DataIntegrityViolationException;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Optional;
import java.util.Set;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.never;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

@ExtendWith(MockitoExtension.class)
class GrowthDiscoveryServiceTest {

    @Mock
    private PostRepository postRepository;
    @Mock
    private PostDisinterestRepository postDisinterestRepository;
    @Mock
    private DailyCheckInRepository dailyCheckInRepository;
    @Mock
    private PointsHistoryRepository pointsHistoryRepository;
    @Mock
```

```
private RewardItemRepository rewardItemRepository;
@Mock
private RedemptionRequestRepository redemptionRequestRepository;
@Mock
private TagRepository tagRepository;
@Mock
private UserRepository userRepository;
@Mock
private CommentRepository commentRepository;
@Mock
private RecommendationConfigRepository recommendationConfigRepository;
@Mock
private NotificationService notificationService;
@Mock
private com.swjtu.forum.repository.UserFollowRepository userFollowRepository;
@Mock
private BadgeService badgeService;

private GrowthDiscoveryService growthDiscoveryService;
private StubUserService userService;
private User user;
private Tag javaTag;

@BeforeEach
void setUp() {
    userService = new StubUserService();
    growthDiscoveryService = new GrowthDiscoveryService(
            userService,
            postRepository,
            postDisinterestRepository,
            dailyCheckInRepository,
            pointsHistoryRepository,
            rewardItemRepository,
            redemptionRequestRepository,
            tagRepository,
            userRepository,
            commentRepository,
            recommendationConfigRepository,
            notificationService,
            userFollowRepository,
            badgeService
    );

    javaTag = new Tag();
    javaTag.setId(1L);
    javaTag.setName("Java");
```

```
        user = new User();
        user.setId(10L);
        user.setUsername("alice");
        user.setTags(Set.of(javaTag));
        userService.setUser(user);
    }

    @Test
    void recommendPostsAddsExplainabilityReasonsFromWeightedSignals() {
        RecommendationConfig config = new RecommendationConfig();
        config.setId(1L);
        config.setTagWeight(60);
        config.setEngagementWeight(25);
        config.setRecencyWeight(15);
        config.setExplainabilityEnabled(true);

        when(recommendationConfigRepository.findById(1L)).thenReturn(java.util.Optional.of(co」
        ↪  nfig));

        Post matched = buildPost(1L, "Java 帖子", 20, 5, 3, 1, 100,
        ↪  LocalDateTime.now().minusHours(3), Set.of(javaTag));
        Post fresh = buildPost(2L, "新帖子", 2, 0, 0, 0, 10,
        ↪  LocalDateTime.now().minusHours(1), Set.of());

        when(postRepository.findHotPosts(any(PageRequest.class))).thenReturn(new
        ↪  PageImpl<>(List.of(matched, fresh)));
        when(postRepository.findByStatusOrderByCreatedAtDesc(any(Integer.class),
        ↪  any(PageRequest.class))).thenReturn(new PageImpl<>(List.of(fresh, matched)));
        when(postRepository.recommendByTagIds(any(), any(Boolean.class),
        ↪  any(PageRequest.class))).thenReturn(new PageImpl<>(List.of(matched)));
        when(postDisinterestRepository.findPostIdsByUserAndPostIdIn(eq(user),
        ↪  any())).thenReturn(Set.of());

        List<Post> result = growthDiscoveryService.recommendPosts("alice", PageRequest.of(0,
        ↪  10)).getContent();

        assertThat(result).isNotEmpty();
        assertThat(result.get(0).getTitle()).isEqualTo("Java 帖子");
        assertThat(result.get(0).getRecommendationReasons()).contains("匹配你的兴趣标签",
        ↪  "近期互动热度较高");
        assertThat(result.get(0).getRecommendationScore()).isNotNull();
        verify(postDisinterestRepository, never()).existsByUserAndPost(any(User.class),
        ↪  any(Post.class));
    }

    @Test
    void recommendPostsFiltersIgnoredPostsWithOneBatchLookup() {
```

```
        RecommendationConfig config = new RecommendationConfig();
        config.setId(1L);
        config.setTagWeight(60);
        config.setEngagementWeight(25);
        config.setRecencyWeight(15);
        config.setExplainabilityEnabled(false);

        when(recommendationConfigRepository.findById(1L)).thenReturn(java.util.Optional.of(co」
        ↪  nfig));

        Post matched = buildPost(1L, "Java 帖子", 20, 5, 3, 1, 100,
        ↪  LocalDateTime.now().minusHours(3), Set.of(javaTag));
        Post ignored = buildPost(2L, "不感兴趣帖子", 30, 5, 3, 1, 100,
        ↪  LocalDateTime.now().minusHours(1), Set.of(javaTag));

        when(postRepository.findHotPosts(any(PageRequest.class))).thenReturn(new
        ↪  PageImpl<>(List.of(matched, ignored)));
        when(postRepository.findByStatusOrderByCreatedAtDesc(any(Integer.class),
        ↪  any(PageRequest.class))).thenReturn(new PageImpl<>(List.of(ignored, matched)));
        when(postRepository.recommendByTagIds(any(), any(Boolean.class),
        ↪  any(PageRequest.class))).thenReturn(new PageImpl<>(List.of(matched, ignored)));
        when(postDisinterestRepository.findPostIdsByUserAndPostIdIn(eq(user),
        ↪  any())).thenReturn(Set.of(ignored.getId()));

        List<Post> result = growthDiscoveryService.recommendPosts("alice", PageRequest.of(0,
        ↪  10)).getContent();

        assertThat(result).extracting(Post::getId).containsExactly(matched.getId());
        verify(postDisinterestRepository, never()).existsByUserAndPost(any(User.class),
        ↪  any(Post.class));
    }

    @Test
    void updateRecommendationConfigRejectsWeightsNotSummingToHundred() {
        RecommendationConfigRequest request = new RecommendationConfigRequest();
        request.setTagWeight(40);
        request.setEngagementWeight(40);
        request.setRecencyWeight(10);
        request.setExplainabilityEnabled(true);

        assertThatThrownBy(() -> growthDiscoveryService.updateRecommendationConfig(request))
                .isInstanceOf(ApiException.class)
                .hasMessage(" 推荐因子权重总和必须为 100");

        verify(recommendationConfigRepository, never()).save(any(RecommendationConfig.class));
    }
```

```
@Test
void searchPostsTreatsHostileKeywordAsBoundQueryData() {
    String hostileKeyword = "' OR 1=1 -- <script>alert(1)</script>";
    PageRequest pageable = PageRequest.of(0, 20);

    when(postRepository.searchPublishedExtended(eq(PostService.STATUS_PUBLISHED),
    ↪  eq(hostileKeyword), eq(pageable)))
            .thenReturn(new PageImpl<>(List.of(), pageable, 0));

    assertThat(growthDiscoveryService.searchPosts("  " + hostileKeyword + "  ",
    ↪  pageable).getContent()).isEmpty();

    verify(postRepository).searchPublishedExtended(PostService.STATUS_PUBLISHED,
    ↪  hostileKeyword, pageable);
}

@Test
void checkInRejectsWhenTodayRecordAlreadyExists() {
    LocalDate today = LocalDate.now();
    DailyCheckIn existing = new DailyCheckIn();
    existing.setUser(user);
    existing.setCheckDate(today);

    when(dailyCheckInRepository.findByUserAndCheckDate(user,
    ↪  today)).thenReturn(Optional.of(existing));

    assertThatThrownBy(() -> growthDiscoveryService.checkIn("alice"))
            .isInstanceOf(ApiException.class)
            .hasMessage("今日已签到");

    verify(dailyCheckInRepository, never()).saveAndFlush(any(DailyCheckIn.class));
}

@Test
void checkInTranslatesUniqueConstraintRaceToDuplicateMessage() {
    LocalDate today = LocalDate.now();
    when(dailyCheckInRepository.findByUserAndCheckDate(user,
    ↪  today)).thenReturn(Optional.empty());
    when(dailyCheckInRepository.saveAndFlush(any(DailyCheckIn.class)))
            .thenThrow(new DataIntegrityViolationException("duplicate check-in"));

    assertThatThrownBy(() -> growthDiscoveryService.checkIn("alice"))
            .isInstanceOf(ApiException.class)
            .hasMessage("今日已签到");

    verify(userRepository, never()).save(any(User.class));
    verify(pointsHistoryRepository, never()).save(any(PointsHistory.class));
```

```
}

@Test
void checkInAwardsPointsAndResetsConsecutiveDaysWithoutYesterday() {
    LocalDate today = LocalDate.now();
    DailyCheckIn savedRecord = new DailyCheckIn();
    savedRecord.setUser(user);
    savedRecord.setCheckDate(today);
    user.setPoints(7);
    user.setExp(3);
    user.setConsecutiveDays(4);

    when(dailyCheckInRepository.findByUserAndCheckDate(user,
    ↪  today)).thenReturn(Optional.empty());
    when(dailyCheckInRepository.saveAndFlush(any(DailyCheckIn.class))).thenReturn(savedRe
    ↪  cord);
    when(dailyCheckInRepository.findByUserAndCheckDate(user,
    ↪  today.minusDays(1))).thenReturn(Optional.empty());

    DailyCheckIn result = growthDiscoveryService.checkIn("alice");

    assertThat(result).isEqualTo(savedRecord);
    assertThat(user.getPoints()).isEqualTo(17);
    assertThat(user.getExp()).isEqualTo(8);
    assertThat(user.getConsecutiveDays()).isEqualTo(1);
    verify(userRepository).save(user);
    verify(badgeService).checkUserBadges(user);

    ArgumentCaptor<PointsHistory> historyCaptor =
    ↪  ArgumentCaptor.forClass(PointsHistory.class);
    verify(pointsHistoryRepository).save(historyCaptor.capture());
    assertThat(historyCaptor.getValue().getChangeType()).isEqualTo("CHECK_IN");
    assertThat(historyCaptor.getValue().getPointsDelta()).isEqualTo(10);
    assertThat(historyCaptor.getValue().getExpDelta()).isEqualTo(5);
}

@Test
void checkInIncrementsConsecutiveDaysWhenYesterdayExists() {
    LocalDate today = LocalDate.now();
    DailyCheckIn savedRecord = new DailyCheckIn();
    DailyCheckIn yesterday = new DailyCheckIn();
    user.setConsecutiveDays(2);

    when(dailyCheckInRepository.findByUserAndCheckDate(user,
    ↪  today)).thenReturn(Optional.empty());
    when(dailyCheckInRepository.saveAndFlush(any(DailyCheckIn.class))).thenReturn(savedRe
    ↪  cord);
```

```
        when(dailyCheckInRepository.findByUserAndCheckDate(user,
        ↪  today.minusDays(1))).thenReturn(Optional.of(yesterday));

        growthDiscoveryService.checkIn("alice");

        assertThat(user.getConsecutiveDays()).isEqualTo(3);
    }

    @Test
    void redeemRejectsWhenStockDepleted() {
        RewardItem reward = reward(5L, " 限量周边", 100);
        when(rewardItemRepository.findById(5L)).thenReturn(java.util.Optional.of(reward));
        when(rewardItemRepository.decrementStock(5L)).thenReturn(0);

        assertThatThrownBy(() -> growthDiscoveryService.redeem(5L, "alice"))
                .isInstanceOf(ApiException.class)
                .hasMessageContaining(" 库存不足");

        verify(userRepository, never()).deductPoints(any(),
        ↪  org.mockito.ArgumentMatchers.anyInt());
    }

    @Test
    void redeemRejectsWhenPointsInsufficient() {
        RewardItem reward = reward(5L, " 限量周边", 100);
        when(rewardItemRepository.findById(5L)).thenReturn(java.util.Optional.of(reward));
        when(rewardItemRepository.decrementStock(5L)).thenReturn(1);
        when(userRepository.deductPoints(10L, 100)).thenReturn(0);

        assertThatThrownBy(() -> growthDiscoveryService.redeem(5L, "alice"))
                .isInstanceOf(ApiException.class)
                .hasMessageContaining(" 积分不足");
    }

    @Test
    void redeemDeductsPointsAtomicallyAndRecordsHistory() {
        RewardItem reward = reward(5L, " 限量周边", 100);
        when(rewardItemRepository.findById(5L)).thenReturn(java.util.Optional.of(reward));
        when(rewardItemRepository.decrementStock(5L)).thenReturn(1);
        when(userRepository.deductPoints(10L, 100)).thenReturn(1);

        User fresh = new User();
        fresh.setId(10L);
        fresh.setUsername("alice");
        fresh.setPoints(50);
        fresh.setExp(20);
        when(userRepository.findById(10L)).thenReturn(java.util.Optional.of(fresh));
```

```
    when(redemptionRequestRepository.save(any(RedemptionRequest.class))).thenAnswer(i ->
    ↪  i.getArgument(0));

    RedemptionRequest saved = growthDiscoveryService.redeem(5L, "alice");

    assertThat(saved.getStatus()).isEqualTo(0);
    assertThat(saved.getPointsCost()).isEqualTo(100);
    verify(rewardItemRepository).decrementStock(5L);
    verify(userRepository).deductPoints(10L, 100);
    verify(pointsHistoryRepository).save(any());
}

private RewardItem reward(Long id, String name, int cost) {
    RewardItem reward = new RewardItem();
    reward.setId(id);
    reward.setName(name);
    reward.setCostPoints(cost);
    reward.setStatus(1);
    reward.setStock(3);
    return reward;
}

private Post buildPost(Long id, String title, int heat, int likes, int comments, int
↪  collects, int views, LocalDateTime createdAt, Set<Tag> tags) {
    Post post = new Post();
    post.setId(id);
    post.setTitle(title);
    post.setHeatScore(heat);
    post.setLikeCount(likes);
    post.setCommentCount(comments);
    post.setCollectCount(collects);
    post.setViewCount(views);
    post.setCreatedAt(createdAt);
    post.setTags(tags);

    Section section = new Section();
    section.setId(20L);
    section.setName(" 课程讨论");
    post.setSection(section);

    User author = new User();
    author.setId(30L);
    author.setUsername("author");
    post.setAuthor(author);
    return post;
}
```

```
    private static class StubUserService extends UserService {
        private User user;

        StubUserService() {
            super(null, null, null, null, null, null);
        }

        void setUser(User user) {
            this.user = user;
        }

        @Override
        public User getUserByUsername(String username) {
            return user;
        }
    }
}
```

2.5 帖子服务测试

```
package com.swjtu.forum.service;

import com.swjtu.forum.dto.CommentRequest;
import com.swjtu.forum.dto.PostRequest;
import com.swjtu.forum.entity.Comment;
import com.swjtu.forum.entity.Post;
import com.swjtu.forum.entity.PostFavorite;
import com.swjtu.forum.entity.PostLike;
import com.swjtu.forum.entity.Section;
import com.swjtu.forum.entity.User;
import com.swjtu.forum.exception.ForbiddenException;
import com.swjtu.forum.exception.NotFoundException;
import com.swjtu.forum.repository.CommentLikeRepository;
import com.swjtu.forum.repository.CommentRepository;
import com.swjtu.forum.repository.PostBrowseHistoryRepository;
import com.swjtu.forum.repository.PostFavoriteRepository;
import com.swjtu.forum.repository.PostLikeRepository;
import com.swjtu.forum.repository.PostRepository;
import com.swjtu.forum.repository.SectionRepository;
import com.swjtu.forum.repository.TagRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Optional;
```

```
import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.never;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

@ExtendWith(MockitoExtension.class)
class PostServiceTest {

    @Mock
    private PostRepository postRepository;

    @Mock
    private SectionRepository sectionRepository;

    @Mock
    private TagRepository tagRepository;

    @Mock
    private CommentRepository commentRepository;

    @Mock
    private PostLikeRepository postLikeRepository;

    @Mock
    private CommentLikeRepository commentLikeRepository;

    @Mock
    private PostFavoriteRepository postFavoriteRepository;

    @Mock
    private PostBrowseHistoryRepository postBrowseHistoryRepository;

    @Mock
    private NotificationService notificationService;

    @Mock
    private BadgeService badgeService;

    private PostService postService;
    private StubUserService userService;
    private StubContentModerationService contentModerationService;
    private User author;
    private User moderator;
    private Section section;
```

```
private Section otherSection;

@BeforeEach
void setUp() {
    userService = new StubUserService();
    contentModerationService = new StubContentModerationService();
    postService = new PostService(
            postRepository,
            sectionRepository,
            tagRepository,
            commentRepository,
            postBrowseHistoryRepository,
            postLikeRepository,
            commentLikeRepository,
            postFavoriteRepository,
            userService,
            contentModerationService,
            notificationService,
            badgeService,
            null
    );

    author = new User();
    author.setId(1L);
    author.setUsername("author");
    author.setRole("USER");

    moderator = new User();
    moderator.setId(2L);
    moderator.setUsername("moderator");
    moderator.setRole("MODERATOR");

    section = new Section();
    section.setId(10L);
    section.setName("Academic");
    section.setModerator(moderator);
    section.setPostCount(0);

    otherSection = new Section();
    otherSection.setId(11L);
    otherSection.setName("Life");
    otherSection.setPostCount(0);
}

@Test
void createPostMarksSensitiveContentAsPending() {
    PostRequest request = new PostRequest();
```

```
    request.setTitle("  敏感测试  ");
    request.setContent("  正文内容  ");
    request.setSectionId(10L);
    userService.setUser(author);
    contentModerationService.setShouldReview(true);

    when(sectionRepository.findById(10L)).thenReturn(Optional.of(section));
    when(postRepository.save(any(Post.class))).thenAnswer(invocation ->
    ↪  invocation.getArgument(0));

    Post created = postService.createPost(request, "author");

    assertThat(created.getStatus()).isEqualTo(PostService.STATUS_PENDING);
    assertThat(created.getTitle()).isEqualTo(" 敏感测试");
    assertThat(created.getContent()).isEqualTo(" 正文内容");
    verify(sectionRepository).save(section);
}

@Test
void updatePostStatusRejectsModeratorOutsideAssignedSection() {
    Post post = new Post();
    post.setId(100L);
    post.setAuthor(author);
    post.setSection(otherSection);
    post.setStatus(PostService.STATUS_PENDING);
    userService.setUser(moderator);

    when(postRepository.findById(100L)).thenReturn(Optional.of(post));

    assertThatThrownBy(() -> postService.updatePostStatus(100L, PostService.STATUS_HIDDEN,
    ↪  "moderator"))
            .isInstanceOf(ForbiddenException.class)
            .hasMessage(" 无权处理该版块的帖子");
}

@Test
void createCommentRejectsNonPublishedPost() {
    Post post = new Post();
    post.setId(200L);
    post.setStatus(PostService.STATUS_PENDING);

    CommentRequest request = new CommentRequest();
    request.setContent(" 一条评论");

    when(postRepository.findById(200L)).thenReturn(Optional.of(post));

    assertThatThrownBy(() -> postService.createComment(200L, request, "author"))
```

```
            .isInstanceOf(ForbiddenException.class)
            .hasMessage(" 当前帖子状态不允许评论");
}

@Test
void getAllPostsDelegatesToRepositoryOrderingQuery() {
    org.springframework.data.domain.Pageable pageable =
    ↪  org.springframework.data.domain.PageRequest.of(0, 20);
    when(postRepository.findAllByOrderByCreatedAtDesc(pageable))
            .thenReturn(org.springframework.data.domain.Page.empty(pageable));

    assertThat(postService.getAllPosts(pageable).getContent()).isEmpty();
    verify(postRepository).findAllByOrderByCreatedAtDesc(pageable);
}

@Test
void getPostByIdHydratesCurrentUserInteractionState() {
    Post post = publishedPost(250L, author);
    post.setViewCount(3);
    User viewer = user("viewer", 5L, "USER");
    userService.setUser(viewer);

    when(postRepository.findById(250L)).thenReturn(Optional.of(post));
    when(postRepository.save(any(Post.class))).thenAnswer(invocation ->
    ↪  invocation.getArgument(0));
    when(postLikeRepository.findByUserAndPost(viewer, post)).thenReturn(Optional.of(new
    ↪  PostLike()));
    when(postFavoriteRepository.findByUserAndPost(viewer,
    ↪  post)).thenReturn(Optional.of(new PostFavorite()));

    Post result = postService.getPostById(250L, "viewer");

    assertThat(result.getLikedByCurrentUser()).isTrue();
    assertThat(result.getFavoritedByCurrentUser()).isTrue();
    assertThat(result.getViewCount()).isEqualTo(4);
}

@Test
void deleteManagedPostDecrementsSectionCountAndDeletesPost() {
    Post post = new Post();
    post.setId(300L);
    post.setAuthor(author);
    post.setSection(section);
    post.setStatus(PostService.STATUS_PUBLISHED);
    post.getTags().add(new com.swjtu.forum.entity.Tag());
    section.setPostCount(5);
```

```
        User admin = new User();
        admin.setId(9L);
        admin.setUsername("admin");
        admin.setRole("ADMIN");
        userService.setUser(admin);

        when(postRepository.findById(300L)).thenReturn(Optional.of(post));

        postService.deleteManagedPost(300L, "admin");

        assertThat(post.getStatus()).isEqualTo(PostService.STATUS_HIDDEN);
        verify(postRepository).save(post);
        verify(postRepository, never()).delete(post);
    }

    @Test
    void deleteManagedPostRejectsModeratorOutsideAssignedSection() {
        Post post = new Post();
        post.setId(301L);
        post.setAuthor(author);
        post.setSection(otherSection);
        userService.setUser(moderator);

        when(postRepository.findById(301L)).thenReturn(Optional.of(post));

        assertThatThrownBy(() -> postService.deleteManagedPost(301L, "moderator"))
                .isInstanceOf(ForbiddenException.class)
                .hasMessage(" 无权处理该版块的帖子");

        verify(postRepository, never()).delete(any(Post.class));
    }

    @Test
    void withdrawOwnedPostHidesAuthorsPost() {
        Post post = new Post();
        post.setId(400L);
        post.setAuthor(author);
        post.setSection(section);
        post.setStatus(PostService.STATUS_PUBLISHED);
        userService.setUser(author);

        when(postRepository.findByIdAndAuthor(400L, author)).thenReturn(Optional.of(post));
        when(postRepository.save(any(Post.class))).thenAnswer(invocation ->
        ↪  invocation.getArgument(0));

        Post withdrawn = postService.withdrawOwnedPost(400L, "author");
```

```
    assertThat(withdrawn.getStatus()).isEqualTo(PostService.STATUS_HIDDEN);
    verify(postRepository).save(post);
}

@Test
void withdrawOwnedPostRejectsNonAuthor() {
    userService.setUser(author);
    when(postRepository.findByIdAndAuthor(401L, author)).thenReturn(Optional.empty());

    assertThatThrownBy(() -> postService.withdrawOwnedPost(401L, "author"))
            .isInstanceOf(NotFoundException.class)
            .hasMessage(" 帖子不存在");
}

@Test
void acceptCommentAllowsAuthorAndClearsPreviousAcceptedComment() {
    User helper = user("helper", 3L, "USER");
    Post post = publishedPost(500L, author);
    Comment previous = comment(10L, post, helper, true);
    Comment target = comment(11L, post, helper, false);
    userService.setUser(author);

    when(postRepository.findById(500L)).thenReturn(Optional.of(post));
    when(commentRepository.findById(11L)).thenReturn(Optional.of(target));
    when(commentRepository.findByPostAndAcceptedTrue(post)).thenReturn(Optional.of(previo」
    ↪ us));
    when(commentRepository.save(any(Comment.class))).thenAnswer(invocation ->
    ↪ invocation.getArgument(0));
    when(postRepository.save(any(Post.class))).thenAnswer(invocation ->
    ↪ invocation.getArgument(0));

    Comment accepted = postService.acceptComment(500L, 11L, "author");

    assertThat(previous.getAccepted()).isFalse();
    assertThat(accepted.getAccepted()).isTrue();
    assertThat(post.getAcceptedCommentId()).isEqualTo(11L);
    assertThat(userService.contributionCount).isEqualTo(10);
}

@Test
void acceptCommentRejectsNonAuthor() {
    User helper = user("helper", 3L, "USER");
    Post post = publishedPost(501L, author);
    userService.setUser(helper);

    when(postRepository.findById(501L)).thenReturn(Optional.of(post));
```

```
    assertThatThrownBy(() -> postService.acceptComment(501L, 12L, "helper"))
            .isInstanceOf(ForbiddenException.class)
            .hasMessage(" 只有楼主可以采纳最佳答案");

    verify(commentRepository, never()).save(any(Comment.class));
}

@Test
void cancelAcceptedCommentClearsPostAcceptedCommentId() {
    Post post = publishedPost(502L, author);
    post.setAcceptedCommentId(13L);
    Comment accepted = comment(13L, post, user("helper", 3L, "USER"), true);
    userService.setUser(author);

    when(postRepository.findById(502L)).thenReturn(Optional.of(post));
    when(commentRepository.findById(13L)).thenReturn(Optional.of(accepted));
    when(commentRepository.save(any(Comment.class))).thenAnswer(invocation ->
    ↪  invocation.getArgument(0));
    when(postRepository.save(any(Post.class))).thenAnswer(invocation ->
    ↪  invocation.getArgument(0));

    Comment cancelled = postService.cancelAcceptedComment(502L, 13L, "author");

    assertThat(cancelled.getAccepted()).isFalse();
    assertThat(post.getAcceptedCommentId()).isNull();
}

private Post publishedPost(Long id, User author) {
    Post post = new Post();
    post.setId(id);
    post.setAuthor(author);
    post.setSection(section);
    post.setStatus(PostService.STATUS_PUBLISHED);
    return post;
}

private Comment comment(Long id, Post post, User author, boolean accepted) {
    Comment comment = new Comment();
    comment.setId(id);
    comment.setPost(post);
    comment.setAuthor(author);
    comment.setStatus(1);
    comment.setAccepted(accepted);
    comment.setContent("comment " + id);
    return comment;
}
```

```
private User user(String username, Long id, String role) {
    User user = new User();
    user.setUsername(username);
    user.setId(id);
    user.setRole(role);
    return user;
}

private static class StubUserService extends UserService {
    private User user;
    private int contributionCount;

    StubUserService() {
        super(null, null, null, null, null, null);
    }

    void setUser(User user) {
        this.user = user;
    }

    @Override
    public User getUserByUsername(String username) {
        return user;
    }

    @Override
    public void addExperience(User user, int exp) {
    }

    @Override
    public void addContribution(User user, int amount) {
        contributionCount += amount;
    }
}

private static class StubContentModerationService extends ContentModerationService {
    private boolean shouldReview;

    void setShouldReview(boolean shouldReview) {
        this.shouldReview = shouldReview;
    }

    @Override
    public boolean shouldReview(PostRequest request) {
        validatePostRequest(request);
        return shouldReview;
    }
```

```
    }
}
```

2.6 帖子控制器测试

```
package com.swjtu.forum.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.swjtu.forum.dto.CommentRequest;
import com.swjtu.forum.dto.PostRequest;
import com.swjtu.forum.entity.Comment;
import com.swjtu.forum.entity.Post;
import com.swjtu.forum.entity.Section;
import com.swjtu.forum.entity.User;
import com.swjtu.forum.exception.GlobalExceptionHandler;
import com.swjtu.forum.service.AnalyticsService;
import com.swjtu.forum.service.GrowthDiscoveryService;
import com.swjtu.forum.service.PostService;
import com.swjtu.forum.service.ReportService;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageImpl;
import org.springframework.data.domain.PageRequest;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.setup.MockMvcBuilders;
import org.springframework.validation.beanvalidation.LocalValidatorFactoryBean;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@ExtendWith(MockitoExtension.class)
class PostControllerTest {

```

```
    private MockMvc mockMvc;
    private final ObjectMapper objectMapper = new ObjectMapper();

    @Mock
    private PostService postService;

    @Mock
    private GrowthDiscoveryService growthDiscoveryService;

    @Mock
    private ReportService reportService;

    @Mock
    private AnalyticsService analyticsService;

    @BeforeEach
    void setUp() {
        LocalValidatorFactoryBean validator = new LocalValidatorFactoryBean();
        validator.afterPropertiesSet();
        PostController controller = new PostController(postService, growthDiscoveryService,
        ↪  reportService, analyticsService);
        mockMvc = MockMvcBuilders.standaloneSetup(controller)
                .setControllerAdvice(new GlobalExceptionHandler())
                .setValidator(validator)
                .build();
    }

    private Post createSamplePost() {
        User author = new User();
        author.setId(1L);
        author.setUsername("author");

        Section section = new Section();
        section.setId(1L);
        section.setName(" 学术交流");

        Post post = new Post();
        post.setId(1L);
        post.setTitle(" 测试帖子");
        post.setContent(" 帖子内容");
        post.setAuthor(author);
        post.setSection(section);
        post.setStatus(1); // PUBLISHED
        post.setLikeCount(0);
        post.setCommentCount(0);
        post.setCollectCount(0);
        post.setCreatedAt(LocalDateTime.now());
```

```
    return post;
}

@Test
void getPostsReturnsPage() throws Exception {
    Post post = createSamplePost();
    Page<Post> page = new PageImpl<>(List.of(post), PageRequest.of(0, 20), 1);

    when(postService.getPosts(any())).thenReturn(page);

    mockMvc.perform(get("/posts"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.code").value(200))
            .andExpect(jsonPath("$.data.content[0].title").value(" 测试帖子"));
}

@Test
void getPostByIdReturnsPost() throws Exception {
    Post post = createSamplePost();

    when(postService.getPostById(eq(1L), any())).thenReturn(post);

    mockMvc.perform(get("/posts/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.code").value(200))
            .andExpect(jsonPath("$.data.title").value(" 测试帖子"));
}

@Test
void getCommentsReturnsPage() throws Exception {
    Comment comment = new Comment();
    comment.setId(1L);
    comment.setContent(" 评论内容");
    Page<Comment> page = new PageImpl<>(List.of(comment), PageRequest.of(0, 20), 1);

    when(postService.getComments(eq(1L), any(), any())).thenReturn(page);

    mockMvc.perform(get("/posts/1/comments"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.code").value(200))
            .andExpect(jsonPath("$.data.content[0].content").value(" 评论内容"));
}

@Test
void getHotPostsReturnsPage() throws Exception {
    Post post = createSamplePost();
    Page<Post> page = new PageImpl<>(List.of(post), PageRequest.of(0, 20), 1);
```

```
        when(postService.getHotPosts(any())).thenReturn(page);

        mockMvc.perform(get("/posts/hot"))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.code").value(200));
    }

    @Test
    void searchPostsReturnsPage() throws Exception {
        Post post = createSamplePost();
        Page<Post> page = new PageImpl<>(List.of(post), PageRequest.of(0, 20), 1);

        when(growthDiscoveryService.searchPosts(eq(" 测试"), any())).thenReturn(page);

        mockMvc.perform(get("/posts/search").param("keyword", " 测试"))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.code").value(200));
    }

    @Test
    void trendingTagsReturnsPublicReadonlyData() throws Exception {
        when(analyticsService.trendingTags()).thenReturn(List.of(Map.of("tag", "考研",
        ↪  "count", 3)));

        mockMvc.perform(get("/posts/trending-tags"))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.code").value(200))
                .andExpect(jsonPath("$.data[0].tag").value(" 考研"));
    }
}
```

2.7 后台管理服务测试

```
package com.swjtu.forum.service;

import com.swjtu.forum.entity.OperationLog;
import com.swjtu.forum.entity.User;
import com.swjtu.forum.exception.ApiException;
import com.swjtu.forum.exception.NotFoundException;
import com.swjtu.forum.repository.CommentRepository;
import com.swjtu.forum.repository.OperationLogRepository;
import com.swjtu.forum.repository.PostRepository;
import com.swjtu.forum.repository.SectionRepository;
import com.swjtu.forum.repository.UserRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
```

```
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.ArgumentCaptor;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Optional;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.never;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

@ExtendWith(MockitoExtension.class)
class AdminOpsServiceTest {

    @Mock
    private PostRepository postRepository;
    @Mock
    private CommentRepository commentRepository;
    @Mock
    private SectionRepository sectionRepository;
    @Mock
    private UserRepository userRepository;
    @Mock
    private OperationLogRepository operationLogRepository;
    @Mock
    private UserService userService;
    @Mock
    private PostService postService;

    private AdminOpsService adminOpsService;

    @BeforeEach
    void setUp() {
        adminOpsService = new AdminOpsService(
                postRepository,
                commentRepository,
                sectionRepository,
                userRepository,
                operationLogRepository,
                userService,
                postService
        );
    }
```

```
@Test
void banUserRejectsMissingUser() {
    when(userRepository.findById(404L)).thenReturn(Optional.empty());

    assertThatThrownBy(() -> adminOpsService.banUser(404L, "admin"))
            .isInstanceOf(NotFoundException.class)
            .hasMessage(" 用户不存在");

    verify(userRepository, never()).save(any(User.class));
    verify(operationLogRepository, never()).save(any(OperationLog.class));
}

@Test
void banUserRejectsLastEnabledAdmin() {
    User admin = new User();
    admin.setId(1L);
    admin.setUsername("admin");
    admin.setRole("ADMIN");
    admin.setEnabled(true);

    when(userRepository.findById(1L)).thenReturn(Optional.of(admin));
    when(userRepository.countByRoleAndEnabledTrue("ADMIN")).thenReturn(1L);

    assertThatThrownBy(() -> adminOpsService.banUser(1L, "admin"))
            .isInstanceOf(ApiException.class)
            .hasMessage(" 至少需要保留一个可用管理员账号");

    verify(userRepository, never()).save(admin);
}

@Test
void banUserDisablesTargetAndWritesOperationLog() {
    User target = new User();
    target.setId(2L);
    target.setUsername("target");
    target.setRole("USER");
    target.setEnabled(true);

    User operator = new User();
    operator.setId(9L);
    operator.setUsername("admin");
    operator.setRole("ADMIN");
    operator.setEnabled(true);

    when(userRepository.findById(2L)).thenReturn(Optional.of(target));
    when(userRepository.save(target)).thenAnswer(invocation -> invocation.getArgument(0));
    when(userService.getUserByUsername("admin")).thenReturn(operator);
```

```
        User saved = adminOpsService.banUser(2L, "admin");

        assertThat(saved.getEnabled()).isFalse();
        verify(userRepository).save(target);

        ArgumentCaptor<OperationLog> logCaptor = ArgumentCaptor.forClass(OperationLog.class);
        verify(operationLogRepository).save(logCaptor.capture());
        assertThat(logCaptor.getValue().getActor()).isEqualTo(operator);
        assertThat(logCaptor.getValue().getAction()).isEqualTo("BAN_USER");
        assertThat(logCaptor.getValue().getTargetType()).isEqualTo("USER");
        assertThat(logCaptor.getValue().getTargetId()).isEqualTo(2L);
    }
}
```

2.8 举报服务测试

```
package com.swjtu.forum.service;

import com.swjtu.forum.dto.BulkReportReviewResponse;
import com.swjtu.forum.dto.BulkReviewReportRequest;
import com.swjtu.forum.dto.CreateReportRequest;
import com.swjtu.forum.dto.ReviewReportRequest;
import com.swjtu.forum.entity.OperationLog;
import com.swjtu.forum.entity.Post;
import com.swjtu.forum.entity.Report;
import com.swjtu.forum.entity.ReportReviewLog;
import com.swjtu.forum.entity.User;
import com.swjtu.forum.exception.ApiException;
import com.swjtu.forum.exception.NotFoundException;
import com.swjtu.forum.repository.OperationLogRepository;
import com.swjtu.forum.repository.PostRepository;
import com.swjtu.forum.repository.ReportRepository;
import com.swjtu.forum.repository.ReportReviewLogRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.ArgumentCaptor;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.data.domain.PageImpl;
import org.springframework.data.domain.PageRequest;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Optional;
```

```
import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.never;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

@ExtendWith(MockitoExtension.class)
class ReportServiceTest {

    @Mock
    private ReportRepository reportRepository;
    @Mock
    private PostRepository postRepository;
    @Mock
    private OperationLogRepository operationLogRepository;
    @Mock
    private ReportReviewLogRepository reportReviewLogRepository;
    @Mock
    private NotificationService notificationService;

    private ReportService reportService;
    private StubUserService userService;
    private User reporter;
    private User reviewer;
    private Post post;

    @BeforeEach
    void setUp() {
        userService = new StubUserService();
        reportService = new ReportService(reportRepository, postRepository,
        ↪  operationLogRepository, reportReviewLogRepository, userService,
        ↪  notificationService);

        reporter = new User();
        reporter.setId(1L);
        reporter.setUsername("reporter");

        reviewer = new User();
        reviewer.setId(2L);
        reviewer.setUsername("moderator");
        reviewer.setRole("MODERATOR");

        post = new Post();
        post.setId(10L);
        post.setTitle(" 帖子");
```

```
    post.setCommentCount(6);
}

@Test
void createPostReportRejectsMissingPost() {
    CreateReportRequest request = new CreateReportRequest();
    request.setReason(" 垃圾广告");
    userService.setUser(reporter);

    when(postRepository.findById(99L)).thenReturn(Optional.empty());

    assertThatThrownBy(() -> reportService.createPostReport(99L, request, "reporter"))
            .isInstanceOf(NotFoundException.class)
            .hasMessage(" 帖子不存在");

    verify(reportRepository, never()).save(any(Report.class));
}

@Test
void createPostReportRejectsInvalidReason() {
    CreateReportRequest request = new CreateReportRequest();
    request.setReason(" 无效原因");
    userService.setUser(reporter);

    when(postRepository.findById(10L)).thenReturn(Optional.of(post));

    assertThatThrownBy(() -> reportService.createPostReport(10L, request, "reporter"))
            .isInstanceOf(ApiException.class)
            .hasMessage(" 举报原因不合法");

    verify(reportRepository, never()).existsByReporterAndPostAndStatus(any(User.class),
    ↪  any(Post.class), any());
    verify(reportRepository, never()).save(any(Report.class));
}

@Test
void createPostReportRejectsSelfReport() {
    CreateReportRequest request = new CreateReportRequest();
    request.setReason(" 垃圾广告");
    post.setAuthor(reporter);
    userService.setUser(reporter);

    when(postRepository.findById(10L)).thenReturn(Optional.of(post));

    assertThatThrownBy(() -> reportService.createPostReport(10L, request, "reporter"))
            .isInstanceOf(ApiException.class)
            .hasMessage(" 不能举报自己的帖子");
```

```
    verify(reportRepository, never()).save(any(Report.class));
}

@Test
void createPostReportRejectsDuplicatePendingReport() {
    CreateReportRequest request = new CreateReportRequest();
    request.setReason(" 骚扰辱骂");
    post.setAuthor(otherUser(3L, "author"));
    userService.setUser(reporter);

    when(postRepository.findById(10L)).thenReturn(Optional.of(post));
    when(reportRepository.existsByReporterAndPostAndStatus(reporter, post,
    ↪  Report.STATUS_PENDING)).thenReturn(true);

    assertThatThrownBy(() -> reportService.createPostReport(10L, request, "reporter"))
            .isInstanceOf(ApiException.class)
            .hasMessage(" 你已举报过该帖子, 请等待处理");

    verify(reportRepository, never()).save(any(Report.class));
}

@Test
void createPostReportTrimsReasonAndDetailBeforeSavingPendingReport() {
    CreateReportRequest request = new CreateReportRequest();
    request.setReason("  虚假信息  ");
    request.setDetail("  证据链接在评论区  ");
    post.setAuthor(otherUser(3L, "author"));
    userService.setUser(reporter);

    when(postRepository.findById(10L)).thenReturn(Optional.of(post));
    when(reportRepository.existsByReporterAndPostAndStatus(reporter, post,
    ↪  Report.STATUS_PENDING)).thenReturn(false);
    when(reportRepository.save(any(Report.class))).thenAnswer(invocation ->
    ↪  invocation.getArgument(0));

    Report saved = reportService.createPostReport(10L, request, "reporter");

    assertThat(saved.getReporter()).isEqualTo(reporter);
    assertThat(saved.getPost()).isEqualTo(post);
    assertThat(saved.getReason()).isEqualTo(" 虚假信息");
    assertThat(saved.getDetail()).isEqualTo(" 证据链接在评论区");
    assertThat(saved.getStatus()).isEqualTo(Report.STATUS_PENDING);
}

@Test
void listReportsFiltersByReasonAndSortsByPriority() {
```

```
    Report older = buildPendingReport(1L, "垃圾广告", LocalDateTime.now().minusHours(20),
    ↪  null);
    Report newerWithDetail = buildPendingReport(2L, "垃圾广告",
    ↪  LocalDateTime.now().minusHours(5), "有补充说明");

    when(reportRepository.findAll(any(org.springframework.data.jpa.domain.Specification.c」
    ↪  lass), eq(PageRequest.of(0, 10))))
            .thenReturn(new PageImpl<>(List.of(newerWithDetail, older), PageRequest.of(0,
            ↪  10), 2));
    when(reportReviewLogRepository.findByReportOrderByCreatedAtDesc(any(Report.class))).t」
    ↪  henReturn(List.of());

    List<Report> result = reportService.listReports(0, "垃圾广告", "priority",
    ↪  PageRequest.of(0, 10)).getContent();

    assertThat(result).hasSize(2);
    assertThat(result.get(0).getId()).isEqualTo(1L);
    assertThat(result.get(1).getId()).isEqualTo(2L);
}

@Test
void reviewReportSavesHistoryAndOperationLog() {
    Report report = buildPendingReport(3L, "违法违规", LocalDateTime.now().minusHours(2),
    ↪  "举报详情");
    userService.setUser(reviewer);

    ReviewReportRequest request = new ReviewReportRequest();
    request.setAccept(true);
    request.setReviewNote(" 属实, 已处理");

    when(reportRepository.findById(3L)).thenReturn(Optional.of(report));
    when(reportRepository.save(any(Report.class))).thenAnswer(invocation ->
    ↪  invocation.getArgument(0));
    when(reportReviewLogRepository.findByReportOrderByCreatedAtDesc(any(Report.class))).t」
    ↪  henReturn(List.of());

    Report saved = reportService.reviewReport(3L, request, "moderator");

    assertThat(saved.getStatus()).isEqualTo(Report.STATUS_ACCEPTED);
    assertThat(saved.getReviewNote()).isEqualTo(" 属实, 已处理");

    ArgumentCaptor<OperationLog> operationLogCaptor =
    ↪  ArgumentCaptor.forClass(OperationLog.class);
    verify(operationLogRepository).save(operationLogCaptor.capture());
    assertThat(operationLogCaptor.getValue().getAction()).isEqualTo("ACCEPT_REPORT");
```

```
    ArgumentCaptor<ReportReviewLog> reviewLogCaptor =
    ↪ ArgumentCaptor.forClass(ReportReviewLog.class);
    verify(reportReviewLogRepository).save(reviewLogCaptor.capture());
    assertThat(reviewLogCaptor.getValue().getAction()).isEqualTo("ACCEPT");
    assertThat(reviewLogCaptor.getValue().getNote()).isEqualTo(" 属实, 已处理");
}

@Test
void acceptingReportHidesTheReportedPost() {
    post.setStatus(PostService.STATUS_PUBLISHED);
    Report report = buildPendingReport(7L, "违法违规", LocalDateTime.now().minusHours(1),
    ↪ null);
    userService.setUser(reviewer);

    ReviewReportRequest request = new ReviewReportRequest();
    request.setAccept(true);

    when(reportRepository.findById(7L)).thenReturn(Optional.of(report));
    when(reportRepository.save(any(Report.class))).thenAnswer(invocation ->
    ↪ invocation.getArgument(0));
    when(reportReviewLogRepository.findByReportOrderByCreatedAtDesc(any(Report.class))).t」
    ↪ henReturn(List.of());

    reportService.reviewReport(7L, request, "moderator");

    assertThat(post.getStatus()).isEqualTo(PostService.STATUS_HIDDEN);
    verify(postRepository).save(post);
}

@Test
void rejectingReportLeavesPostUntouched() {
    post.setStatus(PostService.STATUS_PUBLISHED);
    Report report = buildPendingReport(8L, "违法违规", LocalDateTime.now().minusHours(1),
    ↪ null);
    userService.setUser(reviewer);

    ReviewReportRequest request = new ReviewReportRequest();
    request.setAccept(false);

    when(reportRepository.findById(8L)).thenReturn(Optional.of(report));
    when(reportRepository.save(any(Report.class))).thenAnswer(invocation ->
    ↪ invocation.getArgument(0));
    when(reportReviewLogRepository.findByReportOrderByCreatedAtDesc(any(Report.class))).t」
    ↪ henReturn(List.of());

    reportService.reviewReport(8L, request, "moderator");
```

```
    assertThat(post.getStatus()).isEqualTo(PostService.STATUS_PUBLISHED);
    verify(postRepository, never()).save(any(Post.class));
}

@Test
void bulkReviewRejectsNonPendingReports() {
    Report pending = buildPendingReport(4L, "其他", LocalDateTime.now().minusHours(1),
    ↪  null);
    Report handled = buildPendingReport(5L, "其他", LocalDateTime.now().minusHours(1),
    ↪  null);
    handled.setStatus(Report.STATUS_REJECTED);
    userService.setUser(reviewer);

    BulkReviewReportRequest request = new BulkReviewReportRequest();
    request.setReportIds(List.of(4L, 5L));
    request.setAccept(false);
    request.setReviewNote(" 批量驳回");

    when(reportRepository.findAllById(List.of(4L, 5L))).thenReturn(List.of(pending,
    ↪  handled));

    assertThatThrownBy(() -> reportService.bulkReviewReports(request, "moderator"))
            .isInstanceOf(ApiException.class)
            .hasMessage(" 仅支持批量处理待处理举报");
}

@Test
void bulkReviewProcessesAllPendingReports() {
    Report first = buildPendingReport(6L, "骚扰辱骂", LocalDateTime.now().minusHours(2),
    ↪  null);
    Report second = buildPendingReport(7L, "骚扰辱骂", LocalDateTime.now().minusHours(3),
    ↪  "补充信息");
    userService.setUser(reviewer);

    BulkReviewReportRequest request = new BulkReviewReportRequest();
    request.setReportIds(List.of(6L, 7L));
    request.setAccept(true);
    request.setReviewNote(" 批量采纳");

    when(reportRepository.findAllById(List.of(6L, 7L))).thenReturn(List.of(first,
    ↪  second));
    when(reportRepository.save(any(Report.class))).thenAnswer(invocation ->
    ↪  invocation.getArgument(0));

    BulkReportReviewResponse response = reportService.bulkReviewReports(request,
    ↪  "moderator");
```

```
        assertThat(response.getProcessedCount()).isEqualTo(2);
        assertThat(first.getStatus()).isEqualTo(Report.STATUS_ACCEPTED);
        assertThat(second.getStatus()).isEqualTo(Report.STATUS_ACCEPTED);
        verify(reportReviewLogRepository,
        ↪  org.mockito.Mockito.times(2)).save(any(ReportReviewLog.class));
    }

    private Report buildPendingReport(Long id, String reason, LocalDateTime createdAt, String
    ↪  detail) {
        Report report = new Report();
        report.setId(id);
        report.setReporter(reporter);
        report.setPost(post);
        report.setReason(reason);
        report.setDetail(detail);
        report.setStatus(Report.STATUS_PENDING);
        report.setCreatedAt(createdAt);
        return report;
    }

    private User otherUser(Long id, String username) {
        User user = new User();
        user.setId(id);
        user.setUsername(username);
        return user;
    }

    private static class StubUserService extends UserService {
        private User user;

        StubUserService() {
            super(null, null, null, null, null, null);
        }

        void setUser(User user) {
            this.user = user;
        }

        @Override
        public User getUserByUsername(String username) {
            return user;
        }
    }
}
```

2.9 通知服务测试

```
package com.swjtu.forum.service;

import com.swjtu.forum.entity.Notification;
import com.swjtu.forum.entity.User;
import com.swjtu.forum.exception.NotFoundException;
import com.swjtu.forum.repository.NotificationRepository;
import com.swjtu.forum.repository.UserRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.ArgumentCaptor;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Optional;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.never;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

@ExtendWith(MockitoExtension.class)
class NotificationServiceTest {

    @Mock
    private NotificationRepository notificationRepository;
    @Mock
    private UserRepository userRepository;

    private NotificationService notificationService;
    private User alice;
    private User bob;

    @BeforeEach
    void setUp() {
        notificationService = new NotificationService(notificationRepository, userRepository);
        alice = user(1L, "alice");
        bob = user(2L, "bob");
    }

    @Test
    void notifyOtherSkipsSelfDirectedNotification() {
        notificationService.notifyOther(alice, alice, Notification.TYPE_COMMENT, "t", "c", 9L,
        ↪  "POST");
```

```
    verify(notificationRepository, never()).save(any());
}

@Test
void notifyOtherCreatesNotificationForDifferentRecipient() {
    notificationService.notifyOther(alice, bob, Notification.TYPE_COMMENT, "新评论", "bob
    ↪  评论了你的帖子", 9L, "POST");

    ArgumentCaptor<Notification> captor = ArgumentCaptor.forClass(Notification.class);
    verify(notificationRepository).save(captor.capture());
    Notification saved = captor.getValue();
    assertThat(saved.getRecipient()).isEqualTo(alice);
    assertThat(saved.getType()).isEqualTo(Notification.TYPE_COMMENT);
    assertThat(saved.getIsRead()).isFalse();
    assertThat(saved.getTargetId()).isEqualTo(9L);
}

@Test
void markReadRejectsAnotherUsersNotification() {
    Notification foreign = new Notification();
    foreign.setId(5L);
    foreign.setRecipient(bob);
    when(userRepository.findByUsername("alice")).thenReturn(Optional.of(alice));
    when(notificationRepository.findById(5L)).thenReturn(Optional.of(foreign));

    assertThatThrownBy(() -> notificationService.markRead(5L, "alice"))
            .isInstanceOf(NotFoundException.class);
    verify(notificationRepository, never()).save(any());
}

@Test
void markReadSetsReadFlagForOwner() {
    Notification own = new Notification();
    own.setId(6L);
    own.setRecipient(alice);
    own.setIsRead(false);
    when(userRepository.findByUsername("alice")).thenReturn(Optional.of(alice));
    when(notificationRepository.findById(6L)).thenReturn(Optional.of(own));
    when(notificationRepository.save(any(Notification.class))).thenAnswer(i ->
    ↪  i.getArgument(0));

    Notification result = notificationService.markRead(6L, "alice");

    assertThat(result.getIsRead()).isTrue();
}

private User user(Long id, String username) {
```

```
        User user = new User();
        user.setId(id);
        user.setUsername(username);
        return user;
    }
}
```

C 性能测试脚本

```
import fs from "node:fs";
import os from "node:os";
import path from "node:path";
import { spawn } from "node:child_process";

const root = path.resolve("../../");
const backend = path.join(root, "jiaodaluntan", "backend");
const jar = path.join(backend, "target", "forum-1.0.0.jar");
const port = Number(process.env.PERF_PORT || 18081);
const baseUrl = `http://127.0.0.1:${port}/api`;

const javaArgs = [
  "-jar",
  jar,
  "--spring.profiles.active=demo",
  `--server.port=${port}`,
  "--spring.datasource.url=jdbc:h2:mem:swjtu_perf;MODE=MySQL;DATABASE_TO_LOWER=TRUE;CASE_INSE」
  ↪  NSITIVE_IDENTIFIERS=TRUE;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=FALSE",
  "--demo.seed.enabled=true",
  "--demo.seed.run-on-demo-profile=true",
];

const app = spawn("java", javaArgs, {
  cwd: backend,
  stdio: ["ignore", "ignore", "ignore"],
  windowsHide: true,
});

const sleep = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

async function probe() {
  try {
    const response = await fetch(`${baseUrl}/posts?page=0&size=1`, {
      signal: AbortSignal.timeout(2000),
    });
    return response.status < 500;
  } catch {
```

```
    return false;
  }
}

function percentile(sorted, p) {
  if (!sorted.length) return 0;
  const index = Math.ceil((p / 100) * sorted.length) - 1;
  return sorted[Math.max(0, Math.min(sorted.length - 1, index))];
}

function summarize(name, type, endpointLabel, records, durationMs, concurrency, extra = {}) {
  const ok = records.filter((record) => record.ok);
  const errors = records.length - ok.length;
  const times = ok.map((record) => record.ms).sort((a, b) => a - b);
  const avg = times.reduce((sum, value) => sum + value, 0) / (times.length || 1);
  return {
    name,
    type,
    endpointLabel,
    concurrency,
    durationMs,
    total: records.length,
    success: ok.length,
    errors,
    errorRate: records.length ? errors / records.length : 0,
    throughput: records.length / (durationMs / 1000),
    avg,
    p50: percentile(times, 50),
    p90: percentile(times, 90),
    p95: percentile(times, 95),
    p99: percentile(times, 99),
    max: times[times.length - 1] || 0,
    ...extra,
  };
}

async function oneRequest(url) {
  const start = performance.now();
  try {
    const response = await fetch(baseUrl + url, {
      signal: AbortSignal.timeout(5000),
    });
    await response.arrayBuffer();
    return { ok: response.ok, status: response.status, ms: performance.now() - start };
  } catch (error) {
    return { ok: false, status: 0, ms: performance.now() - start, error: String(error.message
    ↪  || error) };
```

```
  }
}

async function runFixed(name, type, urls, concurrency, seconds, extra = {}) {
  const endAt = Date.now() + seconds * 1000;
  const records = [];
  let index = 0;
  async function worker() {
    while (Date.now() < endAt) {
      records.push(await oneRequest(urls[index++ % urls.length]));
    }
  }
  const start = Date.now();
  await Promise.all(Array.from({ length: concurrency }, worker));
  return summarize(name, type, urls.join(" | "), records, Date.now() - start, concurrency,
  ↪  extra);
}

async function main() {
  let started = false;
  for (let i = 0; i < 90; i += 1) {
    if (await probe()) {
      started = true;
      break;
    }
    await sleep(1000);
  }
  if (!started) {
    app.kill();
    throw new Error("Backend did not start in 90 seconds");
  }

  const mix = [
    "/posts?page=0&size=20",
    "/posts/hot?page=0&size=20",
    "/posts/search?keyword= 测试 &page=0&size=20",
    "/sections?page=0&size=20",
  ];

  const results = [];
  results.push(await runFixed("LOAD-01 常规混合浏览负载", "负载测试", mix, 8, 12, { phase:
  ↪  "baseline" }));
  for (const concurrency of [8, 16, 32, 48]) {
    results.push(await runFixed(
      `STRESS-${concurrency} 并发阶梯`,
      " 压力测试",
      ["/posts/search?keyword= 测试 &page=0&size=20", "/posts?page=0&size=20"],
```

```
    concurrency,
    6,
    { level: concurrency },
  ));
}
for (const pageSize of [10, 20, 50, 100]) {
  results.push(await runFixed(
    `CAP-${pageSize} 分页容量`,
    " 容量测试",
    [`/posts?page=0&size=${pageSize}`,
    ↪  `/posts/search?keyword=测试&page=0&size=${pageSize}`],
    12,
    5,
    { pageSize },
  ));
}
results.push(await runFixed("STAB-01 短时持续混合访问", "稳定性测试", mix, 10, 20, { phase:
↪  "short-soak" }));

app.kill();

const payload = {
  meta: {
    generatedAt: new Date().toISOString(),
    baseUrl,
    host: {
      platform: os.platform(),
      release: os.release(),
      cpus: os.cpus().length,
      totalMemGiB: Number((os.totalmem() / 1024 / 1024 / 1024).toFixed(2)),
    },
    serverInfoFromUser: {
      publicIp: "47.108.218.39",
      privateIp: "172.19.41.159",
      os: "Ubuntu 22.04",
      cpu: "2 核",
      memory: "2 GiB",
      disk: "40 GiB",
      bandwidth: "200 Mbps",
      region: " 西南 1（成都）",
    },
  },
  results,
};

fs.writeFileSync("performance_results.json", JSON.stringify(payload, null, 2), "utf8");
const csv = [
```

```
    "name,type,concurrency,total,success,errors,errorRate,throughput,avg,p50,p90,p95,p99,max,」
    ↪  endpointLabel",
    ...results.map((r) => [
      r.name,
      r.type,
      r.concurrency,
      r.total,
      r.success,
      r.errors,
      r.errorRate.toFixed(4),
      r.throughput.toFixed(2),
      r.avg.toFixed(2),
      r.p50.toFixed(2),
      r.p90.toFixed(2),
      r.p95.toFixed(2),
      r.p99.toFixed(2),
      r.max.toFixed(2),
      `"${r.endpointLabel.replaceAll('"', '""')}"`,
    ].join(",")),
  ];
  fs.writeFileSync("performance_results.csv", csv.join("\n"), "utf8");
  console.log(`Wrote ${results.length} performance result rows.`);
}

main().catch((error) => {
  app.kill();
  console.error(error);
  process.exitCode = 1;
});
```

D 原始结果文件说明

性能测试完整原始数据保存在scripts/performance/performance_results.json。该文件包含生成时间、被测地址、主机信息、接口组合、并发数、持续时间、请求总数、成功数、错误数、吞吐量和各响应时间分位数，可用于复核正文表格与图形。